TECHCRUSH

Cybersecurity Track – Cohort 8 Capstone Project

PROJECT 1

Networking and Securing Data Transmission

Network Design • Virtual Machines • Network Reconnaissance • Packet Analysis
Securing Data Transmission and Monitoring Network Traffic

GROUP 18

Cohort 8

Core Tools: Cisco Packet Tracer, Oracle VirtualBox, Kali Linux, Ubuntu Server, Nmap, Wireshark, OpenSSL, SCP
Group members are listed on the next page.

Group 18 – Project Members

Members who performed the project:

No.	Name	Registration Number
1.	Victoria Oladehinde	CYBE/2026/TC-8/0461
2.	Solomon Ogheneochuko David	CYBE/2026/TC-8/0242
3.	Oriji Kelvin Promise	CYBE/2026/TC-8/0570
4.	Emmanuel Eguzoroh	CYBE/2026/TC-8/0060
5.	Morgan Ekezie	CYBE/2026/TC-8/0140
6.	Amos Adetunlese	CYBE/2026/TC-8/0037
7.	Benjamin Okon Udoh	CYBE/2026/TC-8/0230
8.	Olayinka Aramide Oyinkansola	CYBE/2026/TC-8/0561
9.	Shiaka Paul Woncin	CYBE/2026/TC-8/0536
10.	Adekitan Oluwatunmise E.E	CYBE/2026/TC-8/0255
11.	Anuoluwapo Alabi	CYBE/2026/TC-8/0446
12.	Iyangbe Joshua	CYBE/2026/TC-8/0221
13.	Theresa Nnenna Ndukwe	CYBE/2026/TC-8/0158
14.	Kelvin Enow	CYBE/2026/TC-8/0211
15.	Adewuyi Esther	CYBE/2026/TC-8/0749

Contents

Project 1 – Group 18 Members

Group 18 – Project Members 2

Part A – Networking (Tasks 1–4)

Executive Summary 4

Task 1: Network Design 5

Task 2: Configure Virtual Machines 11

Task 3: Network Reconnaissance 16

Task 4: Packet Analysis 21

Overall Conclusion 26

Part B – Securing Data Transmission and Monitoring Network Traffic

Introduction and Project Objectives 27

Laboratory Environment and Network Configuration 28

Connectivity Verification 30

File Creation, SHA-256 Hash and OpenSSL Encryption 31

Secure File Transfer Using SCP 34

Wireshark Capture and Network Traffic Analysis 36

Verification, Decryption and Final Integrity Check 40

Integrated Security Analysis and Discussion of Results 43

Security Controls, Limitations and Conclusion 45

Executive Summary

This report consolidates four practical networking and cybersecurity tasks into one structured document. Task 1 covers departmental VLAN design in Cisco Packet Tracer. Task 2 establishes a two-machine VirtualBox lab using Kali Linux and Ubuntu Server with SSH connectivity. Task 3 uses Nmap to perform host discovery, TCP port scanning, service-version detection, and operating-system fingerprinting. Task 4 uses Wireshark to capture and analyze ICMP, SSH, and Nmap SYN-scan traffic. Screenshots are retained throughout as the primary evidence that each requirement was completed and verified.

Project Evidence Strategy

- Each task begins with the tutor requirement and the implementation method used to satisfy it.
- Configuration tables and commands are included where they directly support the required deliverable.
- Screenshots are placed beside the relevant explanation so the evidence can be checked immediately.
- Results are summarized using the exact outcomes observed in the lab rather than assumed results.

TASK 1: NETWORK DESIGN

Cisco Packet Tracer departmental VLAN design and connectivity verification

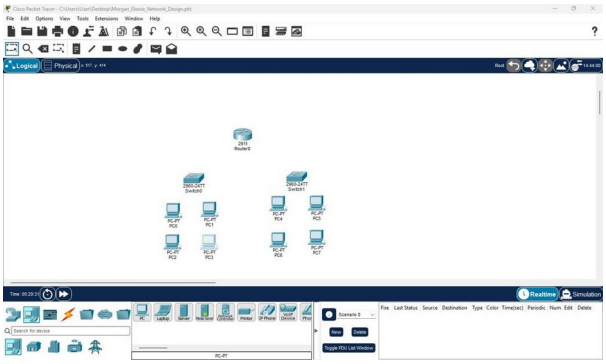
Tutor requirement / design objective

- Build the network in Cisco Packet Tracer using one router, two Cisco 2960 switches, and eight PCs.
- Create four departments with two PCs per department: Administration, Finance, Human Resources, and IT.
- Use VLAN 10 for Administration, VLAN 20 for Finance, VLAN 30 for Human Resources, and VLAN 40 for IT.
- Use the departmental /24 networks 192.168.10.0/24, 192.168.20.0/24, 192.168.30.0/24, and 192.168.40.0/24.
- Devices in the same VLAN must communicate successfully; devices in different VLANs must remain isolated.
- Provide a clear topology, addressing/VLAN information, connectivity evidence, and the saved Packet Tracer project file.

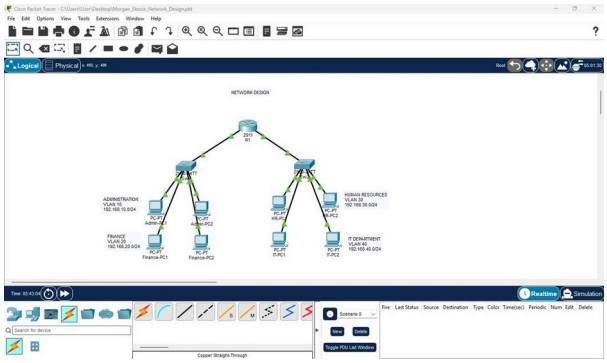
1.1 Implementation Method

1. Placed one Cisco 2911 router, two Cisco 2960 switches, and eight PCs in the logical workspace.
2. Connected the end devices and network devices using the required Ethernet links and organized the topology by department.
3. Named the end devices Admin-PC1/Admin-PC2, Finance-PC1/Finance-PC2, HR-PC1/HR-PC2, and IT-PC1/IT-PC2.
4. Created the required VLANs and assigned each department’s switch access ports to the correct VLAN.
5. Configured static /24 IPv4 addresses on all eight PCs. Default gateways were left unconfigured because inter-VLAN routing was not part of the required connectivity behavior.
6. Verified VLAN membership from the switch CLI and confirmed that the final topology links were operational.
7. Tested same-VLAN communication with successful ping replies and tested cross-VLAN communication with expected timeouts.
8. Saved the completed Cisco Packet Tracer project as Network_Design.pkt.

1.2 Topology Evidence



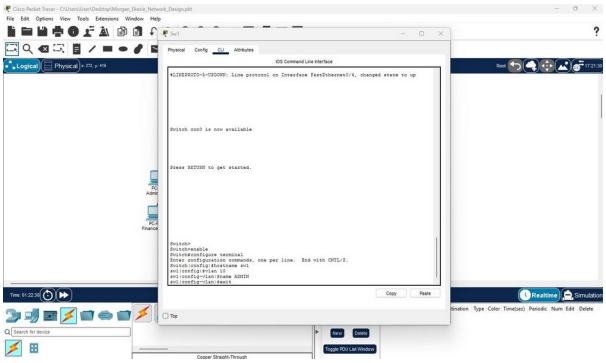
Initial device placement before final cabling and labeling.



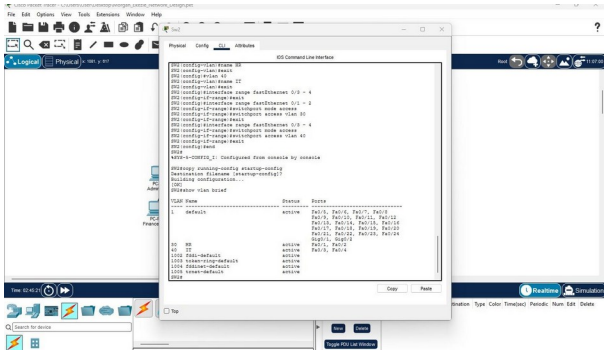
Final labeled departmental topology with VLAN and subnet annotations.

The completed design groups Administration and Finance on the first switch and Human Resources and IT on the second switch. The final labeling makes the VLAN number and subnet for each department visible directly on the topology.

1.3 VLAN and Switch Configuration Evidence



SW1 CLI configuration showing the start of VLAN creation and switch setup.



SW2 verification with VLAN 30 (HR) and VLAN 40 (IT) mapped to their access ports.

The switch configuration used access-mode ports so each departmental PC belonged to only its assigned VLAN. The SW2 verification screenshot explicitly shows VLAN 30 on Fa0/1–Fa0/2 and VLAN 40 on Fa0/3–Fa0/4. The addressing and connectivity tests below confirm the matching configuration across the full topology.

1.4 IP Addressing Table

Device	Department	VLAN	Switch Port	IP Address	Subnet Mask	Gateway
Admin-PC1	Administration	10	SW1 Fa0/1	192.168.10.10	255.255.255.0	N/A

Device	Department	VLAN	Switch Port	IP Address	Subnet Mask	Gateway
Admin-PC2	Administration	10	SW1 Fa0/2	192.168.10.11	255.255.255.0	N/A
Finance-PC1	Finance	20	SW1 Fa0/3	192.168.20.10	255.255.255.0	N/A
Finance-PC2	Finance	20	SW1 Fa0/4	192.168.20.11	255.255.255.0	N/A
HR-PC1	Human Resources	30	SW2 Fa0/1	192.168.30.10	255.255.255.0	N/A
HR-PC2	Human Resources	30	SW2 Fa0/2	192.168.30.11	255.255.255.0	N/A
IT-PC1	IT	40	SW2 Fa0/3	192.168.40.10	255.255.255.0	N/A
IT-PC2	IT	40	SW2 Fa0/4	192.168.40.11	255.255.255.0	N/A

1.5 VLAN Summary

VLAN	Department	Network
10	Administration	192.168.10.0/24
20	Finance	192.168.20.0/24
30	Human Resources	192.168.30.0/24
40	IT	192.168.40.0/24

1.6 Connectivity Verification — Administration and Finance

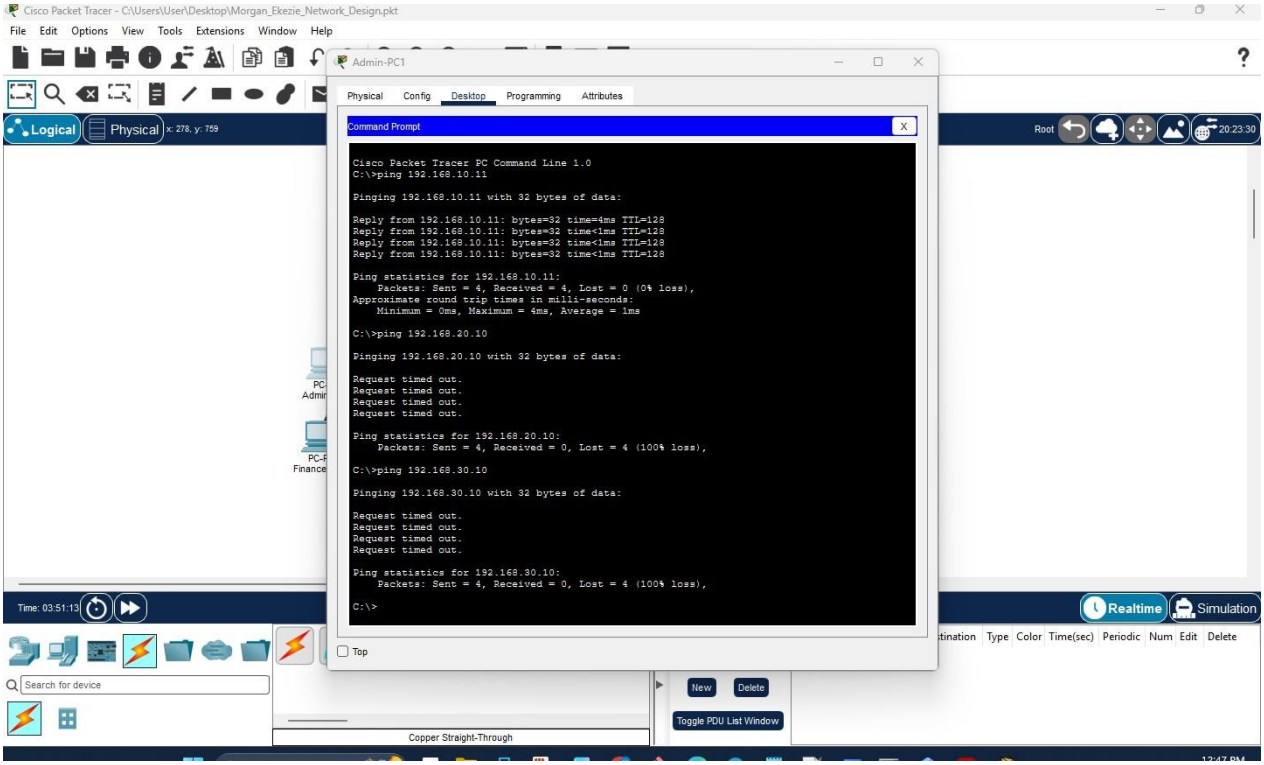


Figure 1.1. Admin-PC1 reaches Admin-PC2 successfully (0% loss), while cross-VLAN pings to Finance and HR time out.

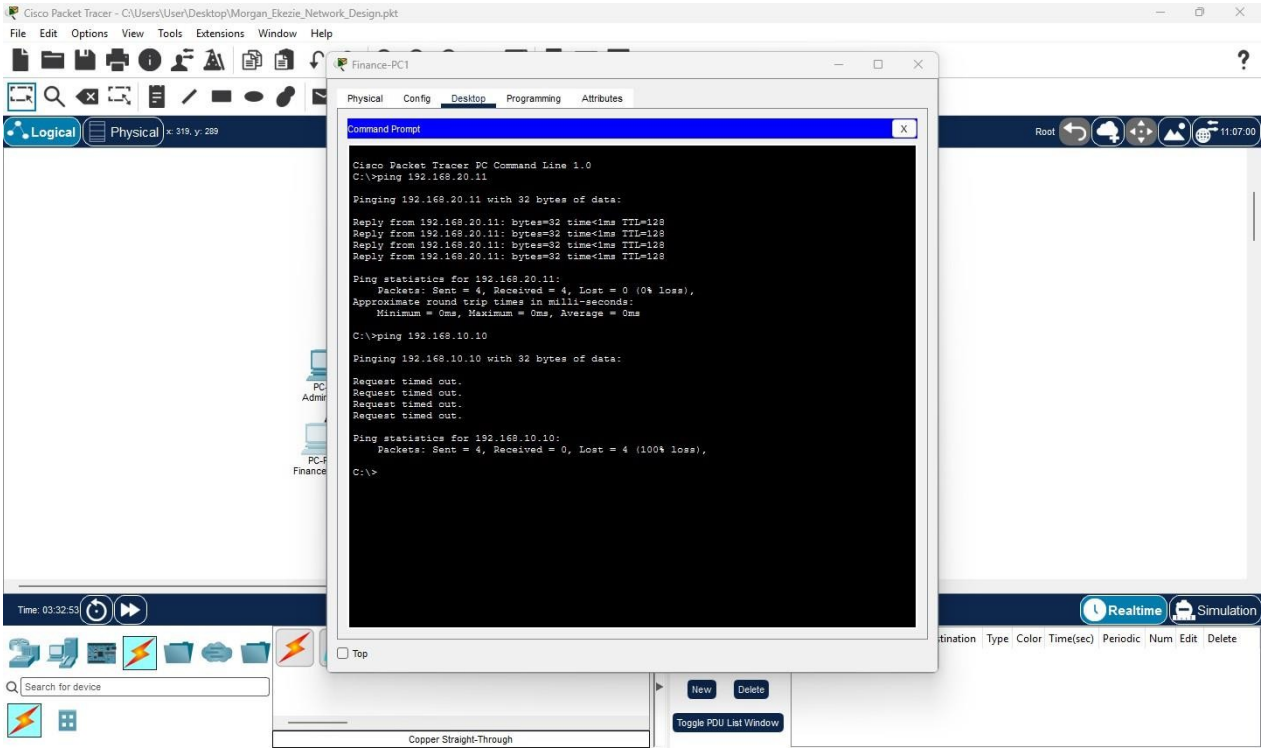


Figure 1.2. Finance-PC1 reaches Finance-PC2 successfully (0% loss), while a cross-VLAN ping to Administration times out.

These results demonstrate the intended VLAN behavior: communication within the same departmental VLAN succeeds, while communication to a different VLAN fails because no inter-VLAN routing/default gateway path was configured.

1.7 Connectivity Verification — Human Resources and IT

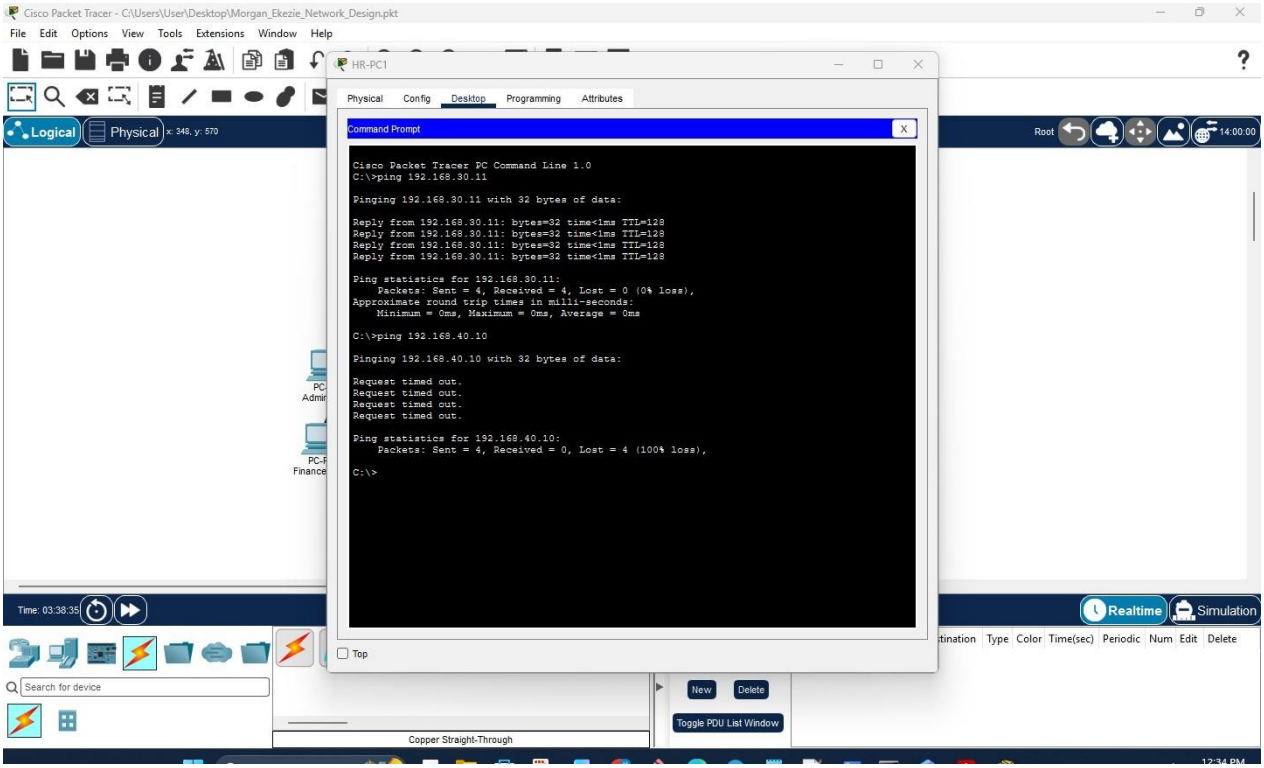


Figure 1.3. HR-PC1 reaches HR-PC2 successfully (0% loss), while the cross-VLAN ping to IT times out.

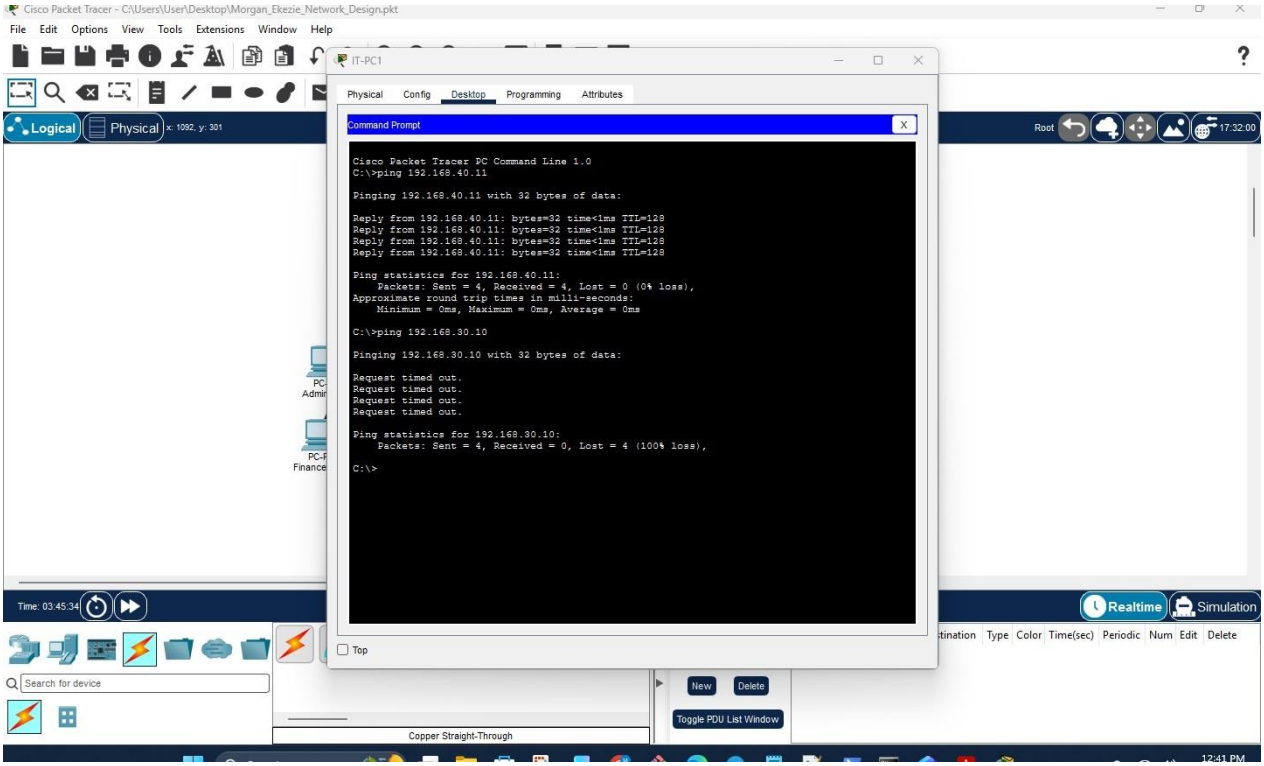


Figure 1.4. IT-PC1 reaches IT-PC2 successfully (0% loss), while the cross-VLAN ping to HR times out.

1.8 Task 1 Result

Requirement	Status	Evidence
Required devices and topology	**PASS**	1 router, 2 switches, and 8 PCs are present and connected.
Four departmental VLANs	**PASS**	VLANs 10, 20, 30, and 40 are represented and configured.
Departmental addressing	**PASS**	Each department uses the required /24 IPv4 network.
Same-VLAN communication	**PASS**	Ping tests return replies with 0% packet loss.
Different-VLAN isolation	**PASS**	Cross-VLAN ping tests time out as required.
Packet Tracer project saved	**PASS**	Project filename visible as Network_Design.pkt.

Task 1 was completed successfully. The final topology, VLAN configuration, addressing plan, and ping evidence collectively demonstrate that the departmental network meets the required segmentation and connectivity behavior.

TASK 2: CONFIGURE VIRTUAL MACHINES

Kali Linux security workstation and Ubuntu Server target in Oracle VirtualBox

Tutor requirement

- Create one Ubuntu VM as the Target and one Kali Linux machine as the Security Workstation.
- Configure both machines on the same network.
- Enable SSH on Ubuntu.
- Verify connectivity using ping and SSH.
- Submit screenshots showing successful connectivity and a network configuration summary.

2.1 Implementation Method

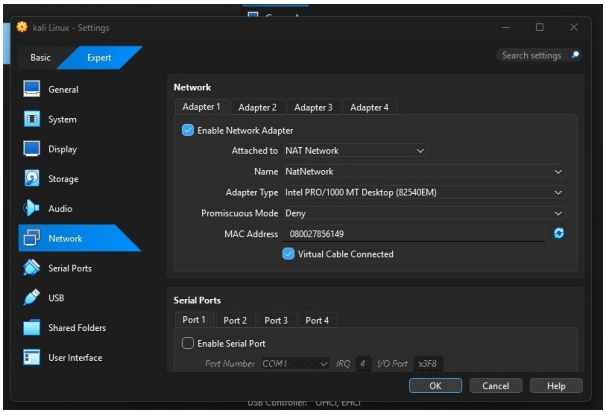
1. Created the Ubuntu-Target virtual machine and used the existing Kali Linux VM as the security workstation.
2. Configured both virtual machines to use the same VirtualBox NAT Network named NatNetwork.
3. Booted both systems and verified their assigned IPv4 addresses using the ip a command.
4. Installed/enabled OpenSSH Server on Ubuntu and verified that ssh.service was enabled and active (running) on TCP port 22.
5. Verified IP connectivity from Kali to Ubuntu using a four-packet ping test.
6. Initiated an SSH session from Kali to Ubuntu and confirmed the remote shell prompt changed to [username]@ubuntu-target.

2.2 Network Configuration Summary

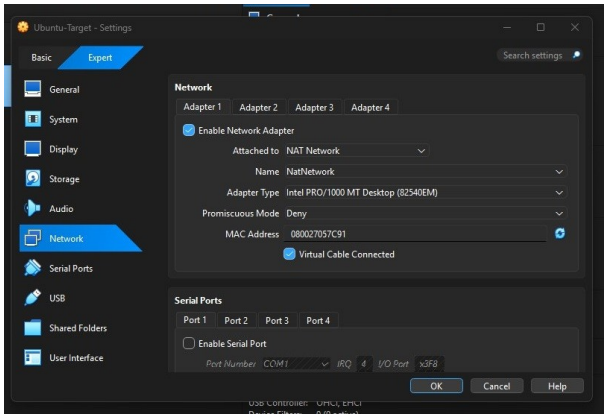
Item	Kali Linux	Ubuntu Target
Role	Security Workstation	Target Machine
Operating System	Kali Linux	Ubuntu Server 26.04.1 LTS
Hypervisor	Oracle VirtualBox	Oracle VirtualBox
Network Mode	NAT Network	NAT Network
NAT Network Name	NatNetwork	NatNetwork
IPv4 Network	10.0.2.0/24	10.0.2.0/24
Network Interface	eth0	enp0s3
IPv4 Address	10.0.2.15/24	10.0.2.4/24
IP Assignment	DHCP	DHCP

Item	Kali Linux	Ubuntu Target
SSH Function	SSH Client	OpenSSH Server
SSH Port	—	TCP 22

2.3 VirtualBox Network Evidence



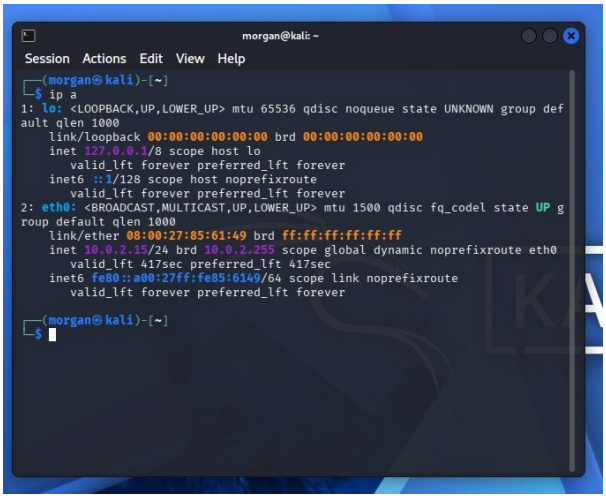
Kali Linux: Adapter 1 attached to NAT Network "NatNetwork".



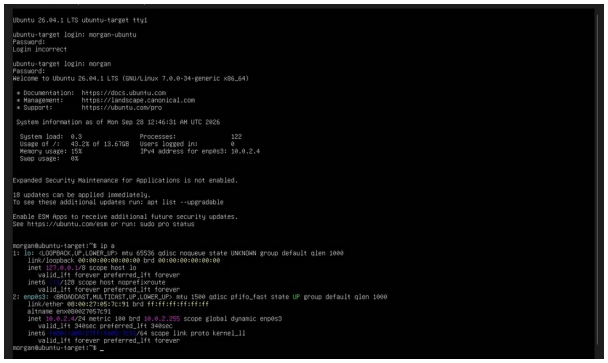
Ubuntu Target: Adapter 1 attached to NAT Network "NatNetwork".

Both adapters are enabled and connected to the same NAT Network, satisfying the shared-network requirement.

2.4 IP Addressing Evidence



Kali Linux eth0: 10.0.2.15/24.



Ubuntu enp0s3: 10.0.2.4/24.

The two systems received different host addresses within the same 10.0.2.0/24 network, allowing direct communication inside the lab.

2.5 SSH Service Evidence

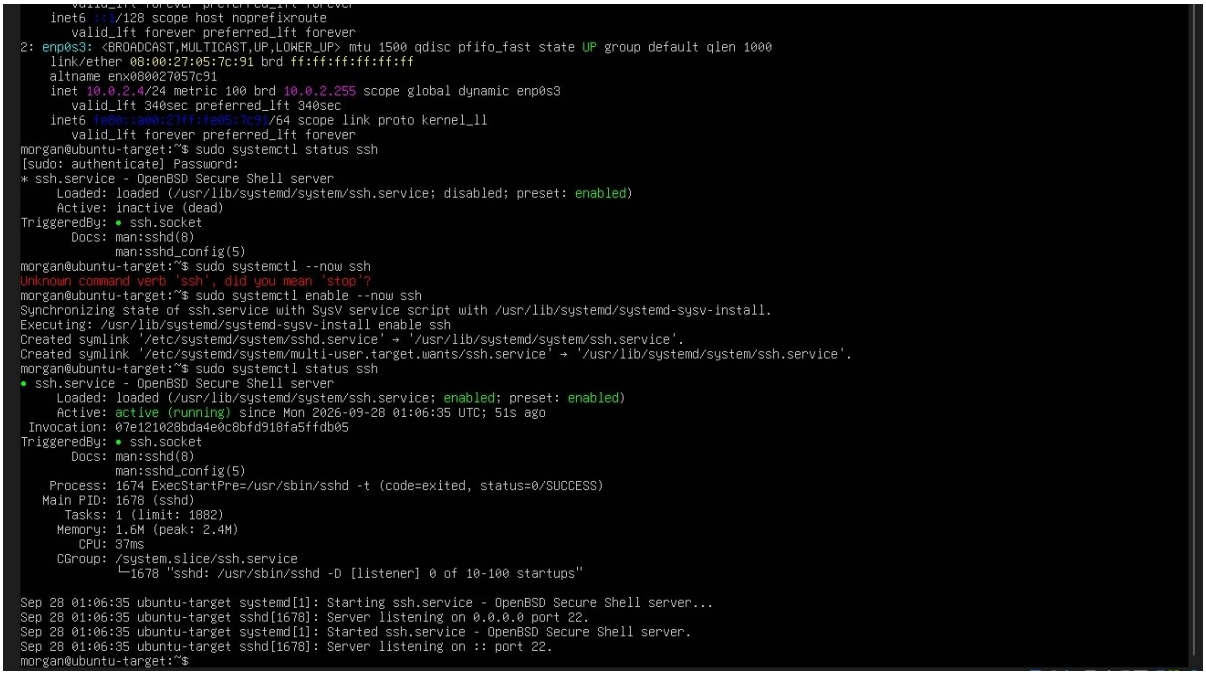


Figure 2.1. Ubuntu ssh.service is enabled and active (running), with the server listening on TCP port 22.

This verifies that the Ubuntu target is configured to accept SSH connections.

2.6 Successful Connectivity Evidence

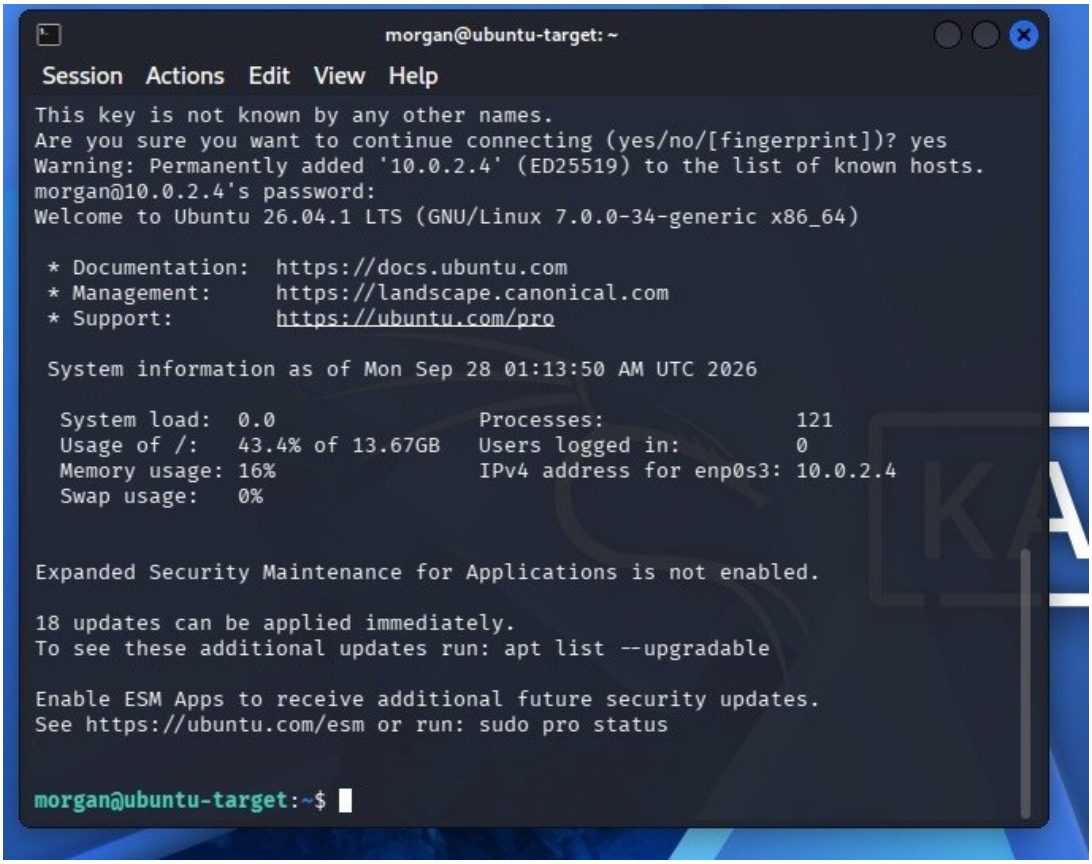


Figure 2.2. Successful SSH login from Kali Linux to Ubuntu; the prompt changes to [username]@ubuntu-target.

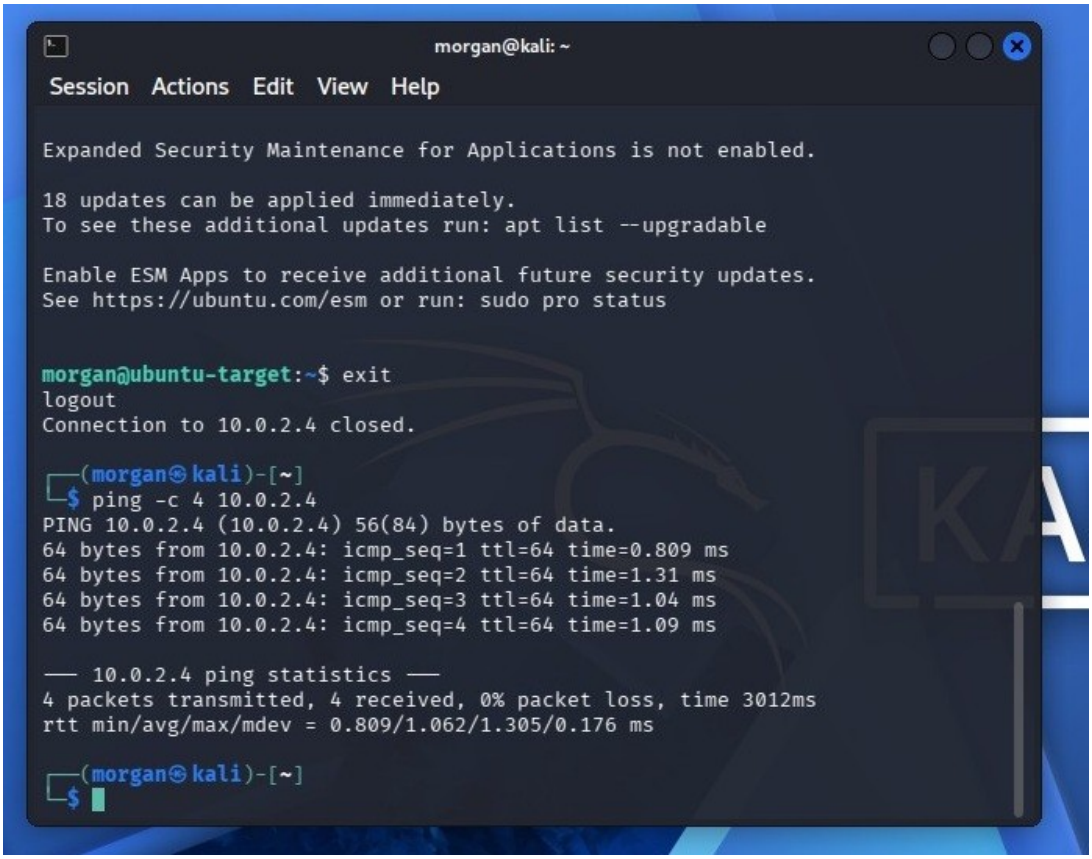


Figure 2.3. Successful ping from Kali to Ubuntu: 4 packets transmitted, 4 received, 0% packet loss.

2.7 Verification Summary

Requirement	Status	Evidence
Same virtual network	**PASS**	Both VMs use NAT Network: NatNetwork.
IPv4 addressing	**PASS**	Kali 10.0.2.15/24; Ubuntu 10.0.2.4/24.
SSH service	**PASS**	Ubuntu ssh.service enabled and active (running).
Ping connectivity	**PASS**	4 transmitted, 4 received, 0% packet loss.
SSH login	**PASS**	Kali successfully logged in as [username]@ubuntu-target.

Task 2 was completed successfully. The screenshots demonstrate the shared VirtualBox network, valid IP addressing, active SSH service, successful ping connectivity, and successful secure remote login.

TASK 3: NETWORK RECONNAISSANCE

Nmap scanning from Kali Linux against the Ubuntu Server target

Tutor requirement

- Using Kali Linux, perform a Ping Scan, TCP Port Scan, Service Version Detection, and Operating System Detection against the Ubuntu Server.
- For each scan, include the command used, a screenshot, and a brief explanation of the findings.
- Summarize the open ports, running services, operating system, potential risks, and two security recommendations.

3.1 Lab Configuration and Scan Plan

Device	Role	IPv4 Address	Network
Kali Linux	Security Workstation	10.0.2.15/24	NatNetwork
Ubuntu Server	Target	10.0.2.4/24	NatNetwork

Scan	Command	Purpose
Ping Scan	nmap -sn 10.0.2.4	Confirm that the target is reachable without scanning ports.
TCP Port Scan	sudo nmap -sS 10.0.2.4	Identify open TCP ports using a SYN scan.
Service Version Detection	sudo nmap -sV 10.0.2.4	Identify services and software versions on open ports.
Operating System Detection	sudo nmap -O 10.0.2.4	Estimate the target OS family using network fingerprinting.

3.2 Ping Scan / Host Discovery

```
nmap -sn 10.0.2.4
```

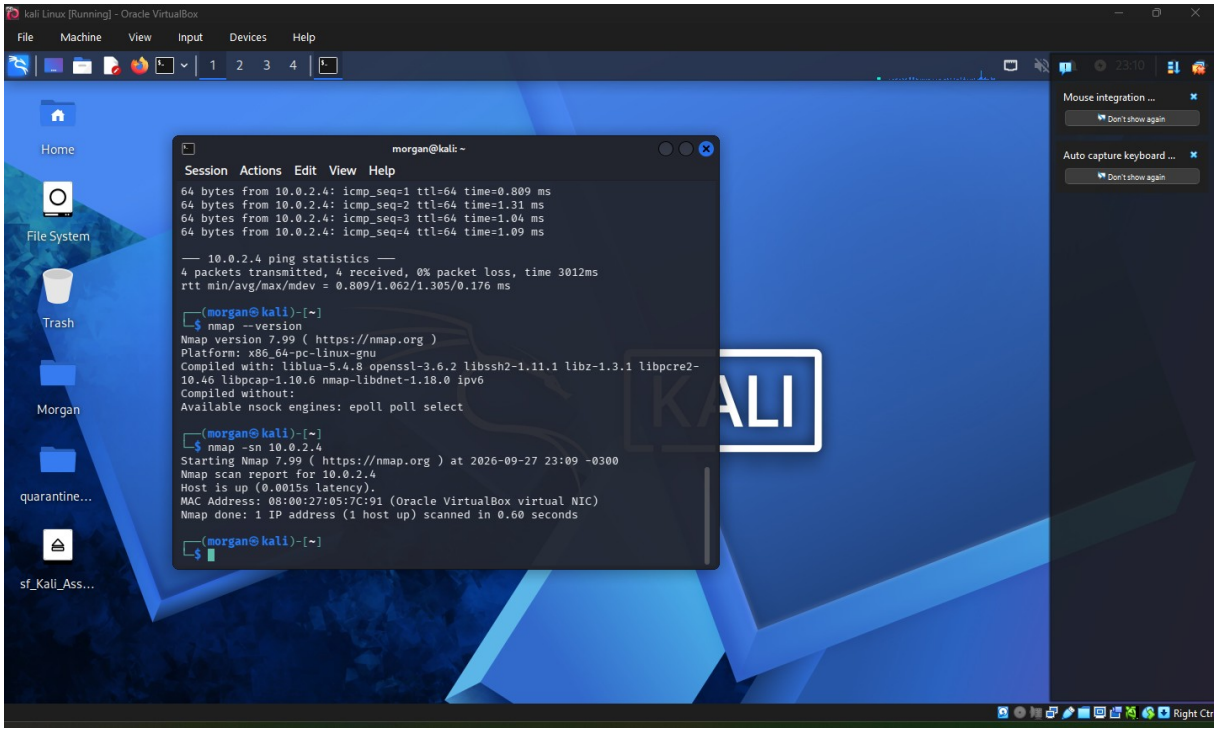


Figure 3.1. Nmap ping scan confirming that the Ubuntu Server at 10.0.2.4 is up and reachable.

Finding: Nmap reported one IP address scanned and one host up. The -sn option performs host discovery without a port scan, so this step confirmed reachability before deeper reconnaissance.

3.3 TCP Port Scan

```
sudo nmap -sS 10.0.2.4
```

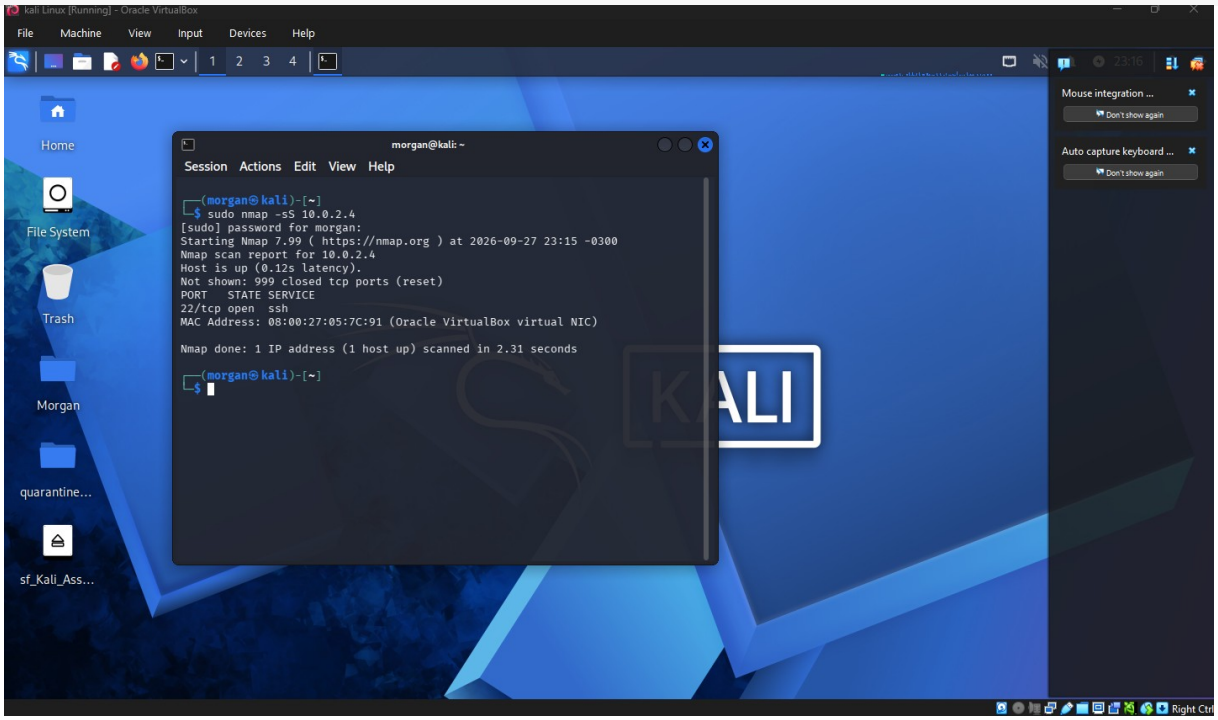


Figure 3.2. TCP SYN scan identifying TCP port 22 (SSH) as the only open port among Nmap's default 1,000 TCP ports.

Finding: Port 22/tcp was open and identified as SSH; the remaining 999 default TCP ports were closed. The -sS option performs a TCP SYN scan and showed that the target exposed only a small number of services.

3.4 Service Version Detection

```
sudo nmap -sV 10.0.2.4
```

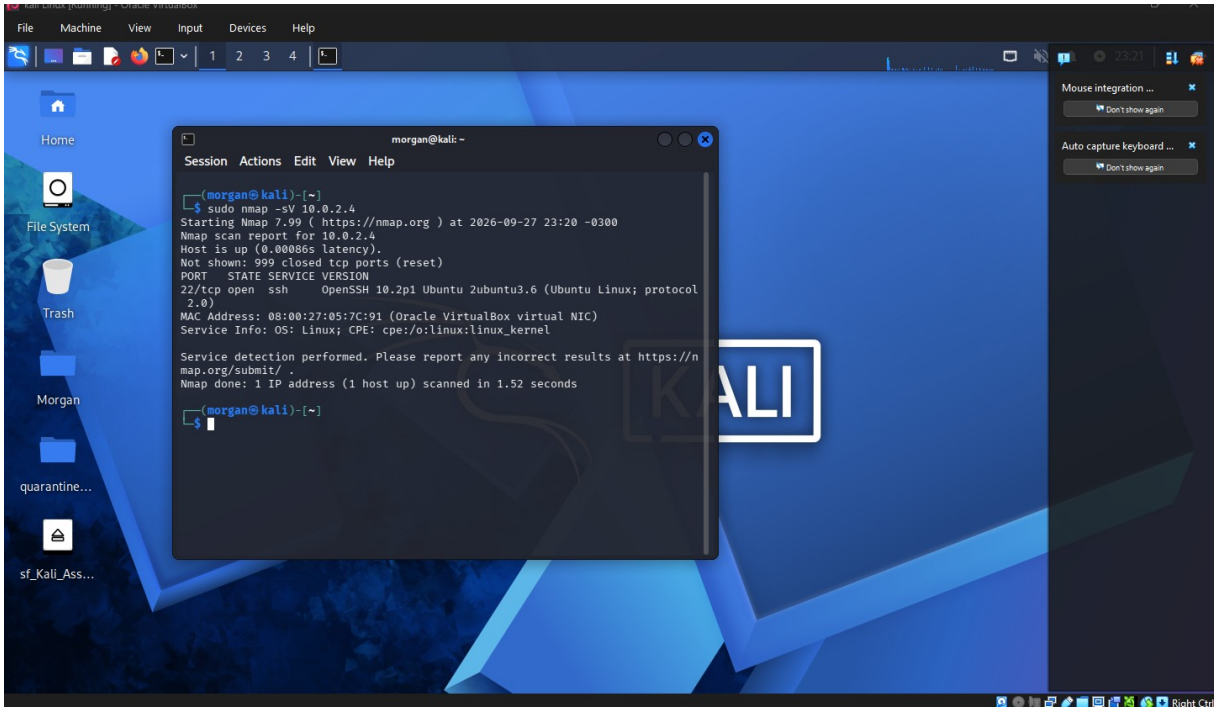


Figure 3.3. Service detection identifying OpenSSH 10.2p1 Ubuntu 2ubuntu3.6 on TCP port 22.

Finding: Nmap identified SSH on port 22 as OpenSSH 10.2p1 Ubuntu 2ubuntu3.6 and returned “Service Info: OS: Linux.” Version detection is useful for inventory and patch management, but exposed version information can also help an attacker profile a target.

3.5 Operating System Detection

```
sudo nmap -O 10.0.2.4
```

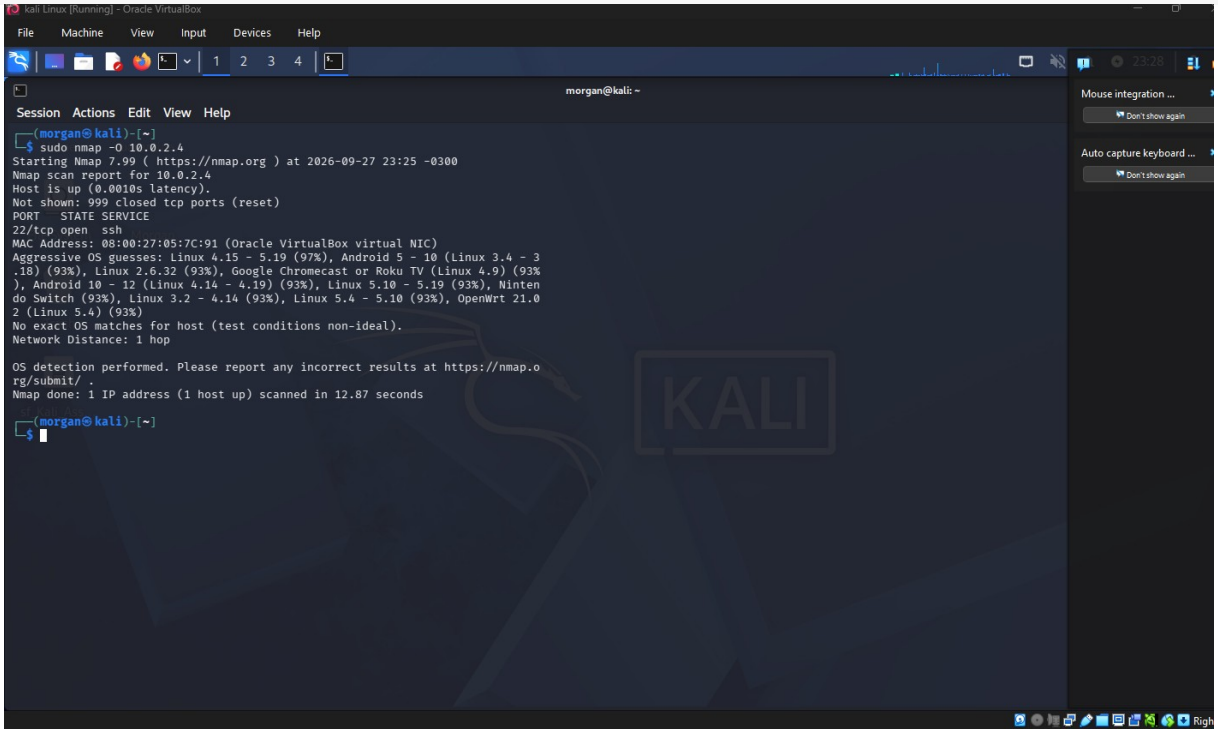


Figure 3.4. OS fingerprinting result showing Linux as the most likely operating-system family.

Finding: Nmap's highest-confidence aggressive guess was Linux 4.15–5.19 at 97%. It also reported that no exact OS match was available because the test conditions were non-ideal. The scan therefore supports the Linux OS family but does not reliably identify the exact Ubuntu release.

3.6 Reconnaissance Findings Summary

Category	Result
Target	Ubuntu Server at 10.0.2.4/24
Host Status	Up and reachable
Open Port	22/tcp
Running Service	SSH
Service Version	OpenSSH 10.2p1 Ubuntu 2ubuntu3.6
OS Family	Linux
Nmap OS Estimate	Linux 4.15–5.19 (97% aggressive guess); no exact OS match
Other Default TCP Ports	999 closed

3.7 Potential Risks

- SSH exposure: TCP port 22 is remotely reachable and can be targeted by credential guessing or unauthorized login attempts if authentication controls are weak.
- Software/version disclosure: service detection reveals the OpenSSH implementation and version, which can help an attacker research version-specific weaknesses if patching is neglected.
- Remote administration impact: compromise of valid SSH credentials could provide command-line access to the server.

3.8 Two Security Recommendations

1. Harden SSH access by using strong credentials and preferably key-based authentication, and restrict SSH access to trusted systems or networks where possible.
2. Keep Ubuntu and OpenSSH fully patched and minimize exposed services so only necessary ports remain reachable.

Task 3 was completed successfully. The reconnaissance identified a reachable Linux target with SSH as the only open default TCP service, providing a clear basis for the risk analysis and security recommendations.

TASK 4: PACKET ANALYSIS

Wireshark capture and analysis of ICMP, SSH, and Nmap scan traffic

Tutor requirement

- Use Wireshark to capture at least three traffic types from ICMP (Ping), DNS, HTTP, SSH, and Nmap Scan.
- For each selected capture, save the capture as a .pcap file, apply an appropriate display filter, and include a screenshot.
- Explain what protocol is being used, what information can be observed, and why the traffic is important.

4.1 Lab Configuration and Capture Summary

Capture	Traffic Generated	Display Filter	Saved File
ICMP Ping	ping -c 4 10.0.2.4	icmp	Task4_ICMP_Ping.pcap
SSH	ssh [username]@10.0.2.4	tcp.port == 22	Task4_SSH.pcap
Nmap Scan	sudo nmap -sS 10.0.2.4	tcp.flags.syn == 1 && tcp.flags.ack == 0	Task4_Nmap_Scan.pcap

Capture interface: eth0 on Kali Linux. Kali Linux used 10.0.2.15 and the Ubuntu Server target used 10.0.2.4 on the VirtualBox NAT Network.

4.2 ICMP (Ping) Capture

Command: ping -c 4 10.0.2.4
Display filter: icmp

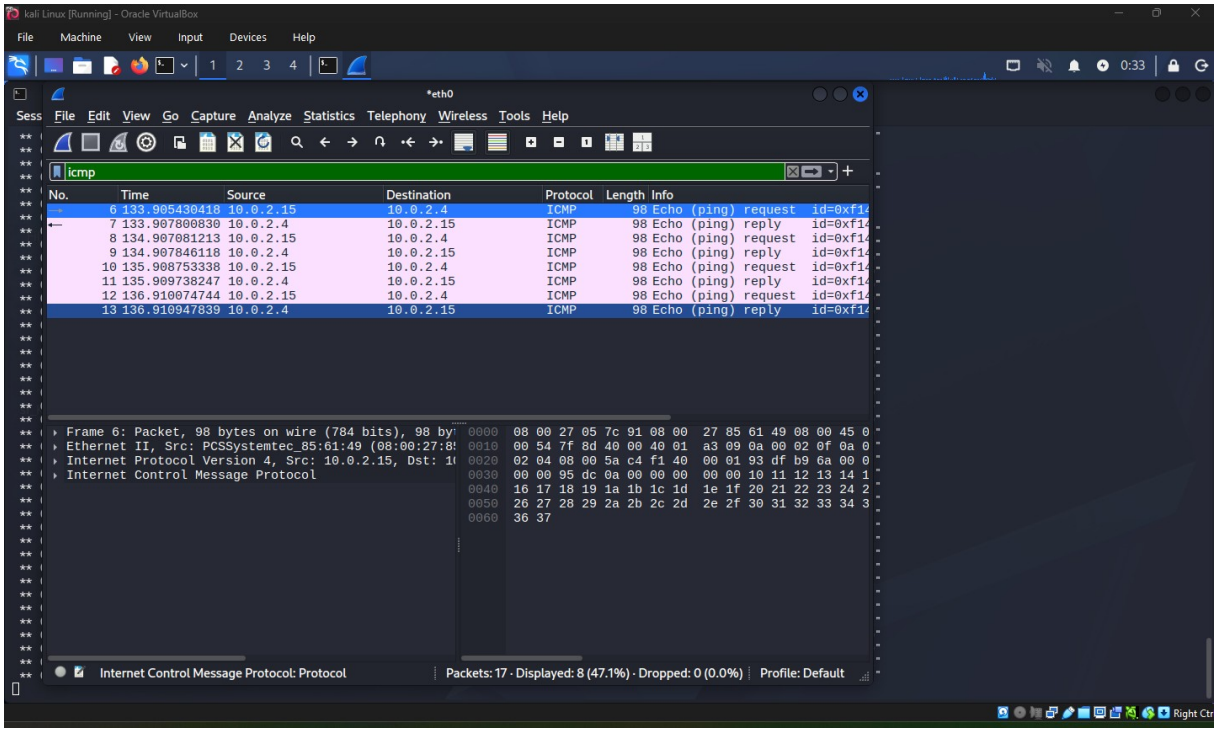


Figure 4.1. ICMP Echo Request and Echo Reply traffic between Kali Linux and the Ubuntu Server.

Protocol used: ICMP (Internet Control Message Protocol).

Information observed: Kali Linux (10.0.2.15) sent four ICMP Echo Request packets to Ubuntu (10.0.2.4), and Ubuntu returned four matching Echo Reply packets. The capture exposes source/destination addresses, message type, protocol, and packet length.

Why this traffic is important: ICMP is used to verify reachability and troubleshoot connectivity. From a security perspective, repeated ICMP probes can also reveal host-discovery activity.

Finding: The Ubuntu Server responded successfully to all observed ICMP Echo Requests.

4.3 SSH Capture

Command: ssh [username]@10.0.2.4
Display filter: tcp.port == 22

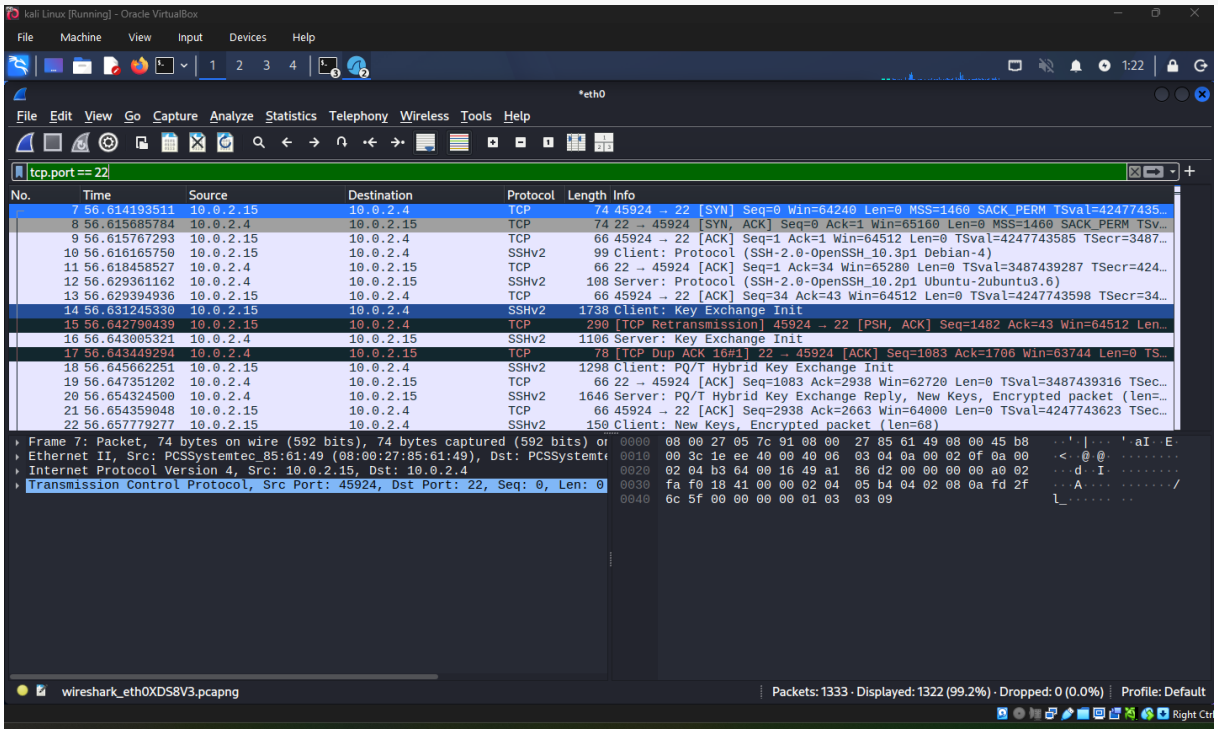


Figure 4.2. SSH traffic on TCP port 22 showing connection setup, protocol negotiation, key exchange, and encrypted packets.

Protocol used: SSHv2 over TCP port 22.

Information observed: The capture shows the TCP three-way handshake (SYN, SYN-ACK, ACK), SSH protocol identification strings, key-exchange messages, endpoint IP addresses/ports, and encrypted SSH application traffic. The password and commands are not visible because the session content is encrypted.

Why this traffic is important: SSH is widely used for secure remote administration. Monitoring SSH traffic helps analysts identify remote-access sessions, connection attempts, unusual source systems, and potential brute-force or unauthorized access activity.

Finding: A successful encrypted SSH session was established from Kali Linux to the Ubuntu Server on TCP port 22.

4.4 Nmap Scan Capture

Command: sudo nmap -sS 10.0.2.4
Display filter: tcp.flags.syn == 1 && tcp.flags.ack == 0

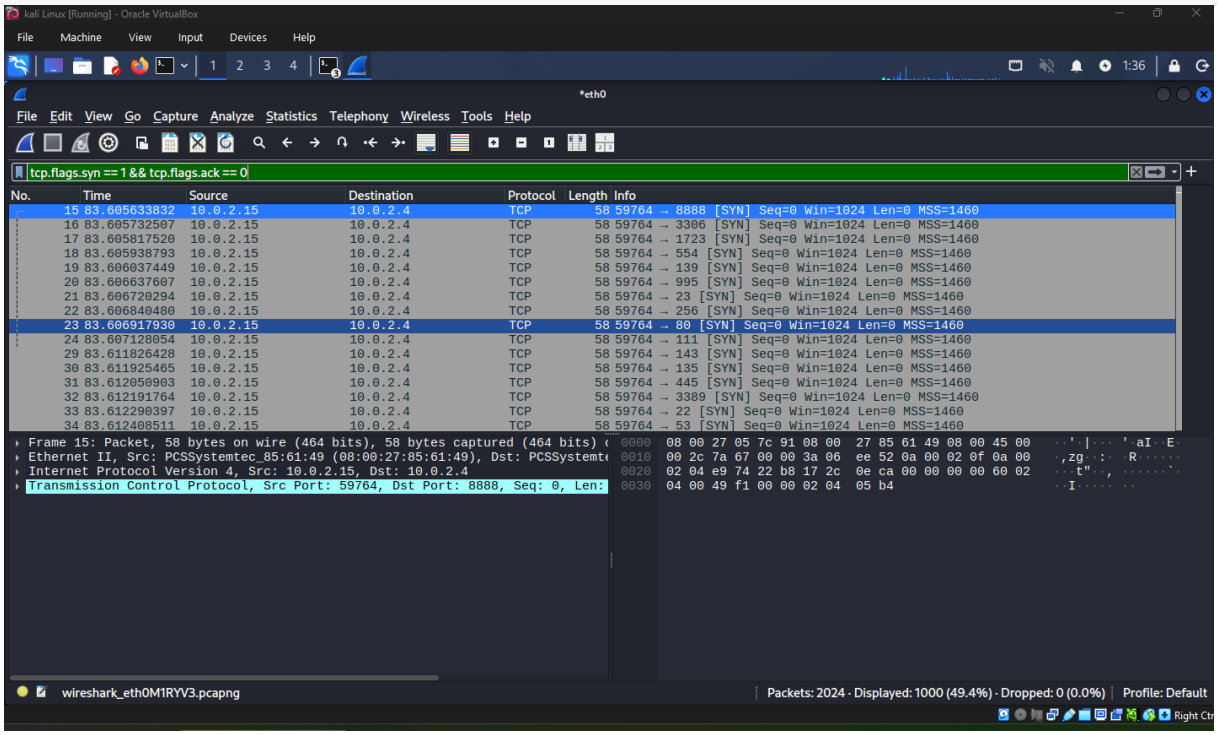


Figure 4.3. Nmap SYN-scan traffic showing repeated SYN probes from Kali Linux to multiple TCP ports on the Ubuntu Server.

Protocol used: TCP, specifically TCP SYN probes generated by an Nmap SYN scan.

Information observed: Kali Linux (10.0.2.15) sent a large number of TCP packets with the SYN flag set to many destination ports on Ubuntu (10.0.2.4). The filter excludes SYN-ACK responses and highlights the initial scan probes.

Why this traffic is important: Rapid SYN probes from one source to many destination ports are a classic reconnaissance pattern. Defenders monitor this behavior because it may indicate vulnerability assessment, inventory activity, or malicious discovery before an intrusion attempt.

Finding: The packet pattern clearly demonstrates automated TCP port reconnaissance from Kali Linux against the Ubuntu target.

4.5 Consolidated Findings

Traffic Type	Main Evidence	Security Meaning
ICMP	Four request/reply pairs between 10.0.2.15 and 10.0.2.4	Confirms reachability; can also reveal host discovery.
SSH	TCP/22 handshake, SSHv2 negotiation, key exchange, encrypted payload	Confirms secure remote administration and confidentiality of session content.
Nmap	Many SYN probes from one source to numerous destination ports	Typical reconnaissance/port-scanning pattern that defenders should monitor.

Task 4 was completed successfully. Wireshark captured three distinct traffic types, each capture was filtered and saved as a separate .pcap file, and the screenshots demonstrate how packet analysis can validate connectivity, inspect encrypted-service behavior, and detect reconnaissance patterns.

Overall Conclusion

Across Tasks 1–4, the project progressed from network construction and segmentation to virtual-lab deployment, active reconnaissance, and packet-level analysis. The Cisco Packet Tracer network demonstrated VLAN-based departmental isolation. The VirtualBox lab established reliable Kali-to-Ubuntu connectivity with SSH. Nmap identified the target's reachable host status, open SSH service, software version, and Linux OS family. Wireshark then provided packet-level evidence of ICMP, SSH, and SYN-scan behavior. Together, the four tasks demonstrate practical understanding of network design, virtualization, Linux administration, reconnaissance, and traffic analysis.

Completed Deliverables

- Task 1: Cisco Packet Tracer topology, addressing/VLAN documentation, VLAN isolation evidence, and saved .pkt project.
- Task 2: VirtualBox network summary plus screenshots proving same-network configuration, SSH service, ping, and SSH connectivity.
- Task 3: Four Nmap reconnaissance scans with commands, screenshots, findings, risks, and two security recommendations.
- Task 4: Three Wireshark captures saved as .pcap files with display filters, screenshots, and protocol/security explanations.

Key Practical Skills Demonstrated

Area	Skills Demonstrated
Network Design	Topology planning, VLAN segmentation, IPv4 addressing, access-port assignment, connectivity testing
Virtualization	VirtualBox VM creation, NAT Network configuration, Linux networking, SSH service management
Reconnaissance	Nmap host discovery, SYN scanning, service-version detection, OS fingerprinting
Packet Analysis	Wireshark capture selection, display filtering, ICMP/SSH/TCP interpretation, scan-pattern recognition

The screenshots throughout this report serve as the primary verification evidence for the completed work.

CYBERSECURITY TRACK – COHORT 8
CAPSTONE PROJECT: NETWORKING AND SECURING DATA TRANSMISSION
Cohort: 8
Project: Securing Data Transmission and Monitoring Network Traffic

1. Introduction

Data transmitted across a network can be exposed to interception, modification, or unauthorized access if appropriate security mechanisms are not used. This practical project demonstrates fundamental techniques for protecting data confidentiality and verifying data integrity during transmission.

This experiment was conducted in a controlled laboratory environment using two Kali Linux virtual machines running in Oracle VirtualBox. The systems were connected through an isolated Internal Network named SecureLab. The practical activities involve encryption and hashing using Linux command-line tools, transfer of files between the two systems using SCP, and monitoring and analysis of the resulting network traffic using Wireshark.

The project focuses on three important security concepts. Encryption protects the confidentiality of information by transforming readable data into an unreadable form. Hashing provides a mechanism for checking whether data has changed by comparing hash values before and after transmission. Finally, secure file transfer using SCP over SSH provides an encrypted communication channel between the sender and receiver.

This project work therefore demonstrates how security controls can be applied to data before, during, and after transmission.

2. Project Objectives

The main objectives of this practical exercise were to:

- Configure two Kali Linux virtual machines for controlled communication.
- Establish communication between the sender and receiver systems.
- Create a controlled file for the transmission experiment.
- Generate a SHA-256 hash to provide an integrity reference.
- Encrypt the file using OpenSSL.
- Transfer files between the two virtual machines using SCP.
- Capture the network traffic generated during transmission using Wireshark.
- Analyse the captured traffic and identify the characteristics of the file-transfer communication.
- Demonstrate the importance of encryption and hashing in secure data transmission.

3. Laboratory Environment

The experiment was performed using two virtual machines:

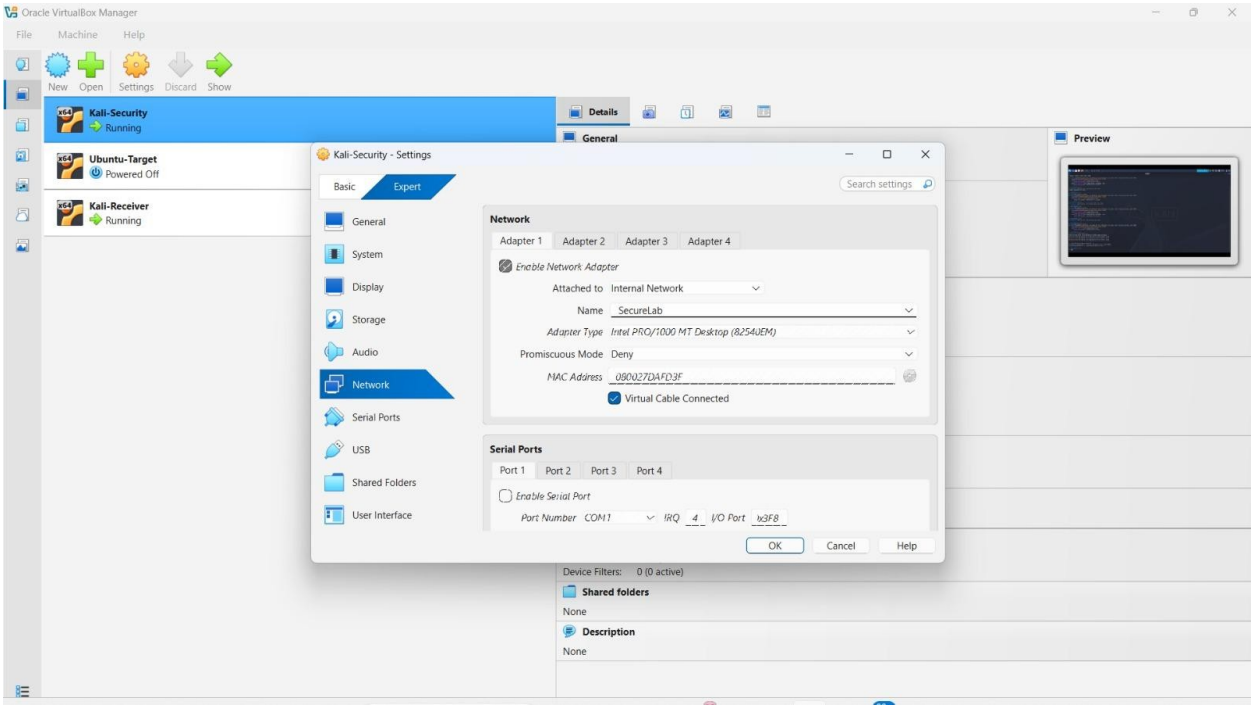
System	Role	IP Addresses
Kali-Security	Sending	192.168.50.20/24
Kali-Receiver	Receiving	192.168.50.10/24

Both systems were connected to the same SecureLab Internal Network in Oracle VirtualBox.

4. Network Configuration

4.1 Establishing the SecureLab Internal Network

Screenshot 1: Configuration of the SecureLab Internal Network

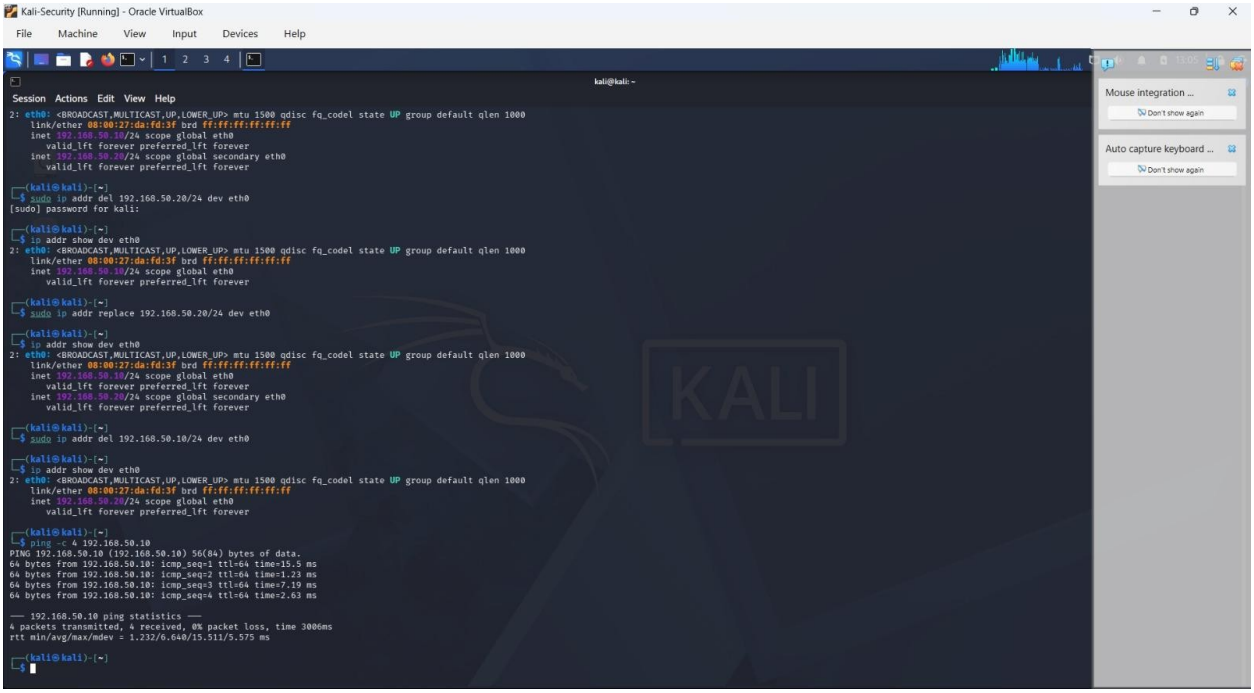


The two Kali Linux virtual machines were configured to communicate through an isolated VirtualBox Internal Network named SecureLab. The network adapter was enabled and the virtual cable was connected. Using an Internal Network provides a controlled laboratory environment in which the virtual machines can communicate directly with one another without relying on an external network connection.

This configuration was used to provide a suitable environment for testing file transmission and observing the resulting network traffic.

4.2 Kali-Security Network Configuration

Figure 2: Kali-Security IP Configuration

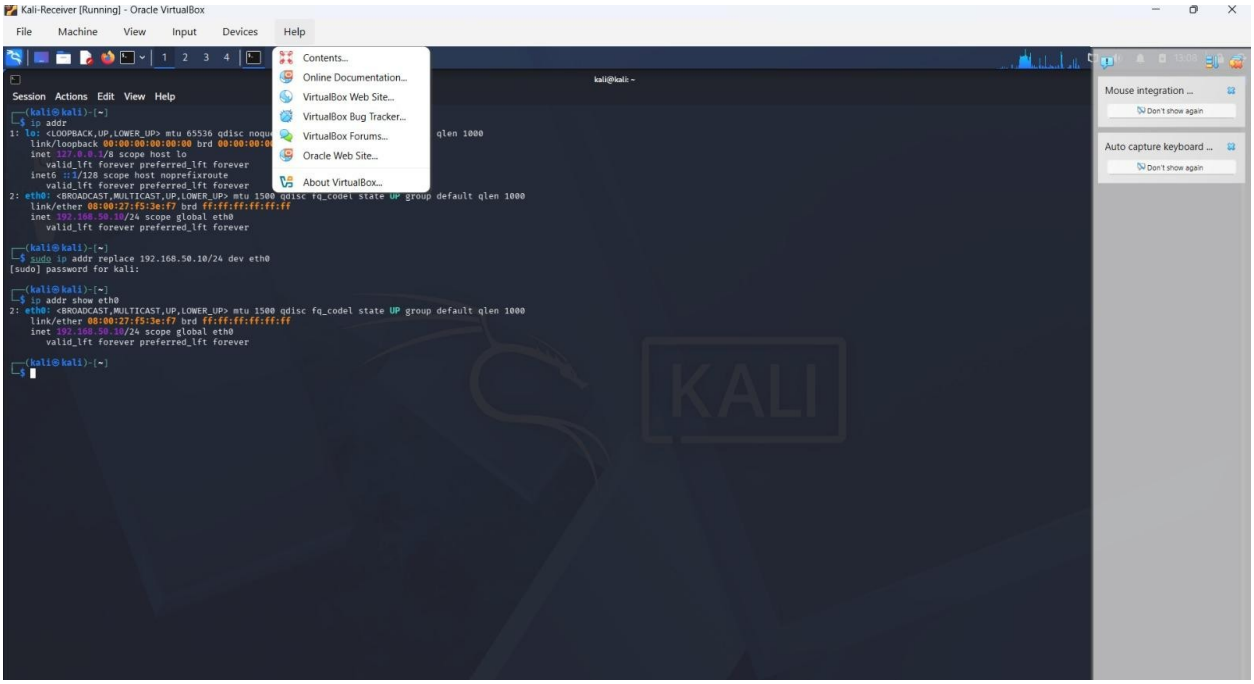


Kali-Security was configured as the sending system for the experiment. Its eth0 interface was assigned the IPv4 address 192.168.50.20/24. The /24 prefix places the system within the 192.168.50.0/24 network.

The configuration was verified using the ip addr show command. This ensured that Kali-Security had a defined address through which it could communicate with the receiver.

4.3 Kali-Receiver Network Configuration

Figure 3: Kali-Receiver IP Configuration

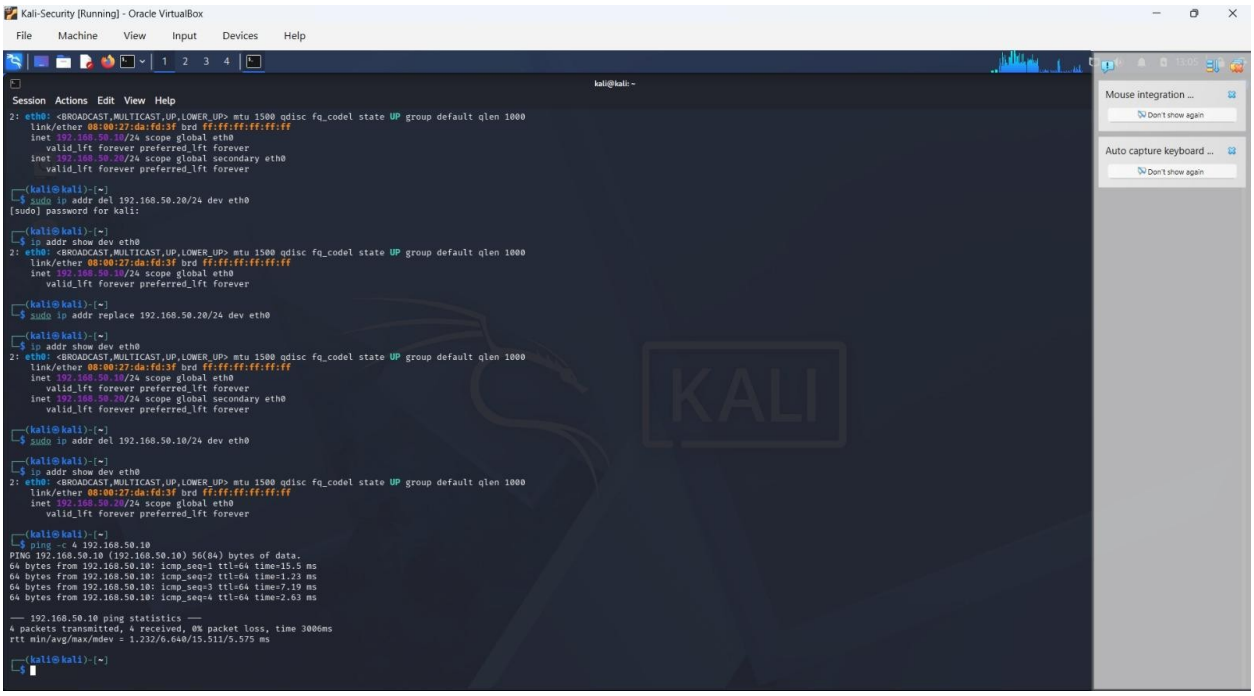


Kali-Receiver was configured as the receiving system. Its eth0 interface was assigned the IPv4 address 192.168.50.10/24. This address is within the same 192.168.50.0/24 network as Kali-Security.

The separate IP addresses allow the two systems to be distinguished during communication and network traffic analysis. The configuration was verified using the ip addr show command.

5. Connectivity Verification

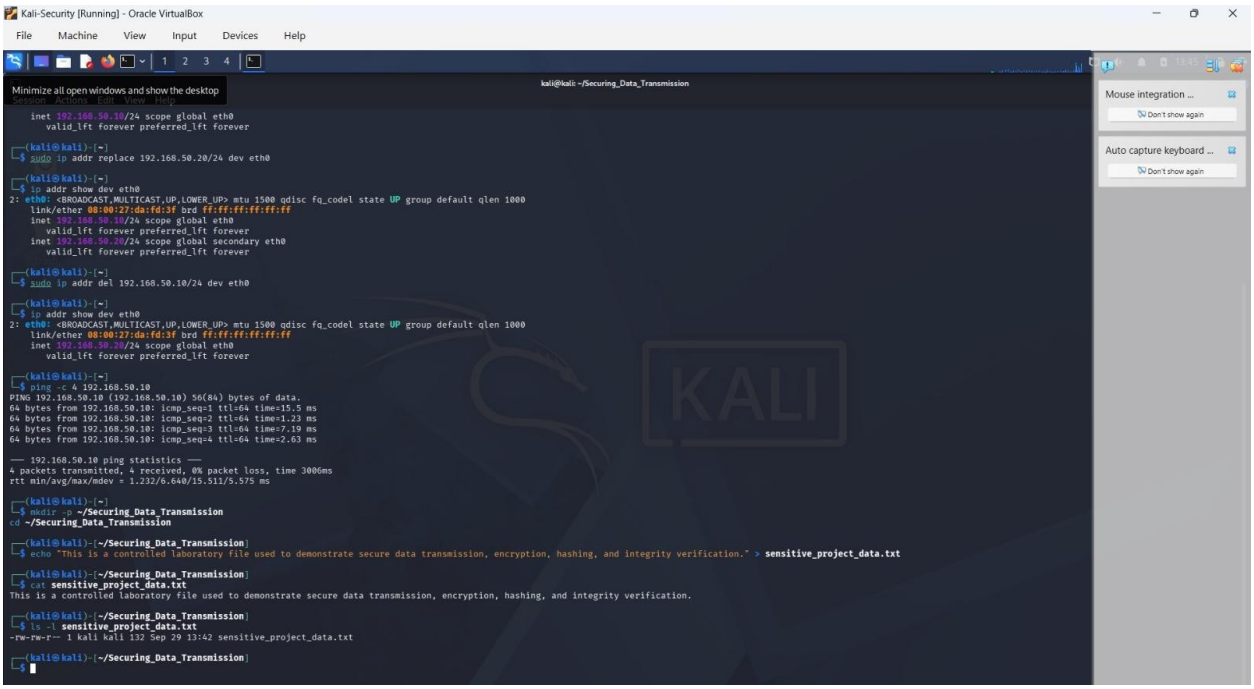
Figure 4: Connectivity Test between Kali-Security and Kali-Receiver Virtual Machines



Connectivity between the two virtual machines was verified using the ping utility. Kali-Security, with IP address 192.168.50.20, sent four ICMP echo requests to Kali-Receiver at 192.168.50.10. The successful responses confirmed that the two systems could communicate through the SecureLab internal network. This verification was completed before commencing the file creation, encryption, hashing, and transfer stages of the experiment.

6. Creation of the Plaintext Test File

Figure 5: Creation and Verification of the Plaintext Test File



A controlled plaintext file named sensitive_project_data.txt was created on the Kali-Security system. The file contains laboratory text specifically prepared for the secure data-transmission experiment.

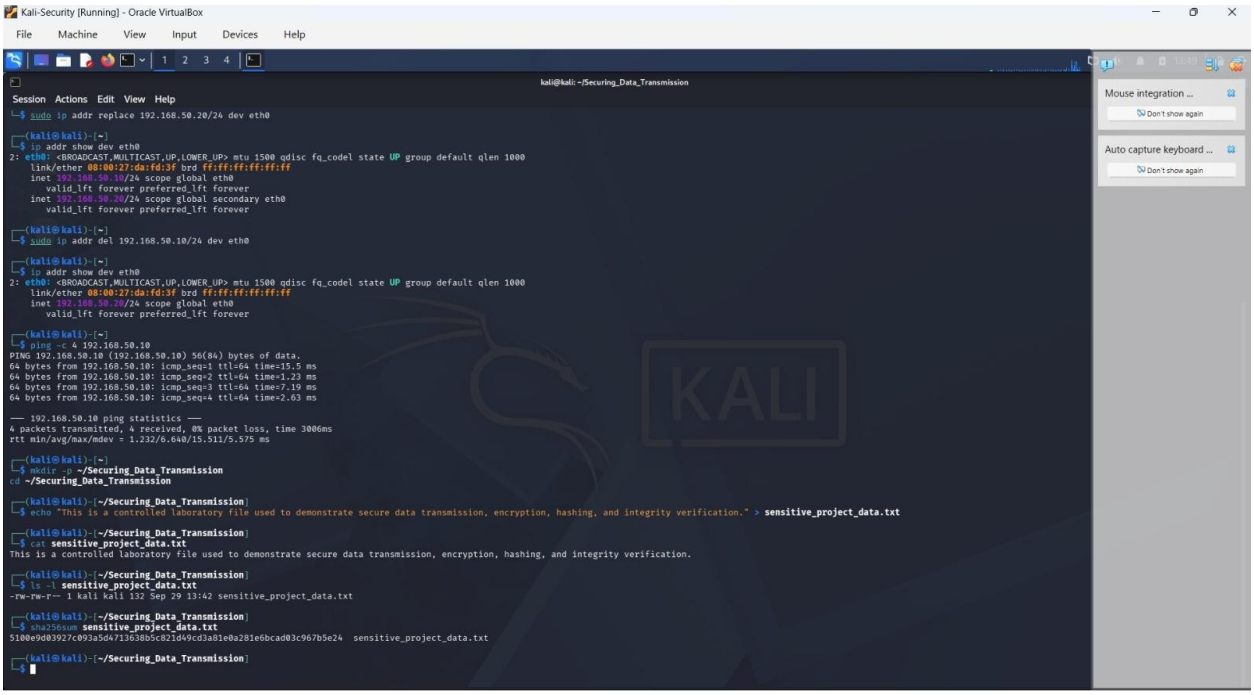
The cat command was used to display the contents of the file and confirm that the expected information had been written successfully. The ls -l command was then used to verify that the file existed in the Securing_Data_Transmission working directory.

This plaintext file represents the original data that will be subjected to hashing and encryption before being transferred between the two virtual machines. Using a controlled test file ensures that the experiment can be performed without exposing real sensitive information.

7. Generate the SHA-256 Hash

Figure 6: Generation of the SHA-256 Integrity Hash

Explanation for the report



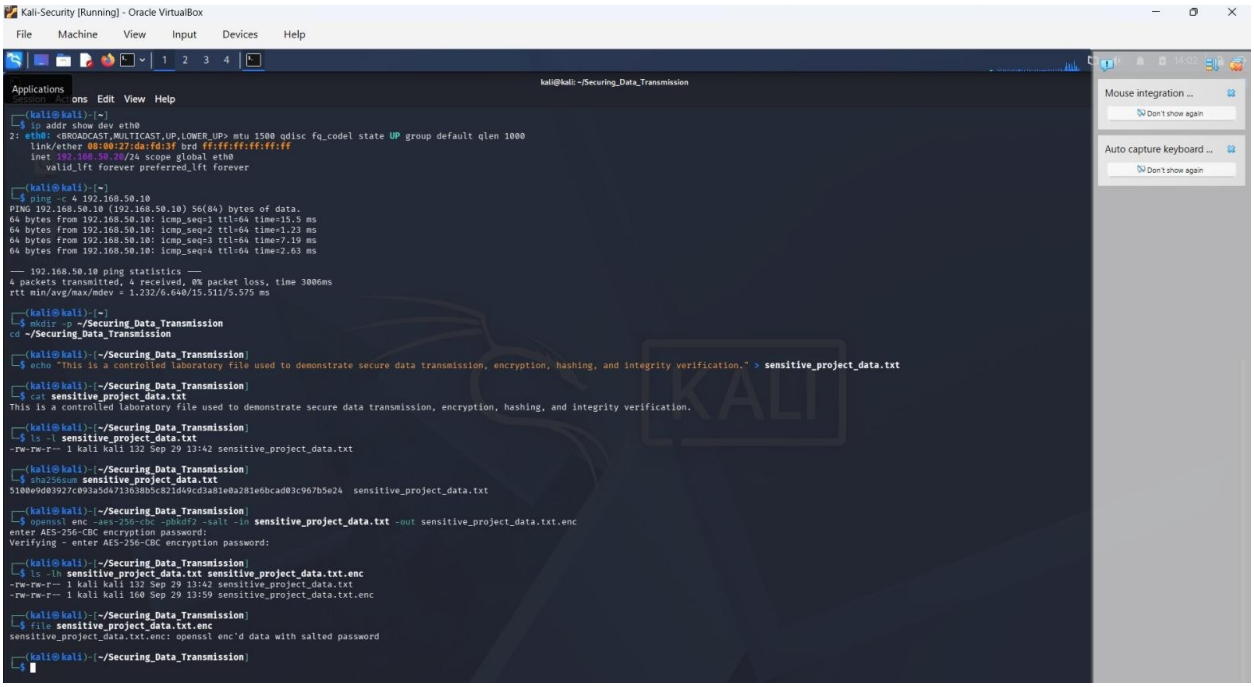
A SHA-256 cryptographic hash was generated for the original plaintext file using the sha256sum command. The resulting hash value provides a unique integrity reference for the file at the time the hash was generated.

The hash does not encrypt or conceal the contents of the file. Instead, it can be used later to determine whether the file has changed. After transmission and subsequent recovery of the file, the SHA-256 value can be calculated again and compared with the original value. Matching values indicate that the file contents remained unchanged during the tested process, while a different value would indicate that the file contents had changed.

The original SHA-256 value will therefore be retained as the reference value for the later integrity verification stage.

8. File Encryption Using OpenSSL

Figure 7: AES-256-CBC Encryption of the Plaintext File Using OpenSSL



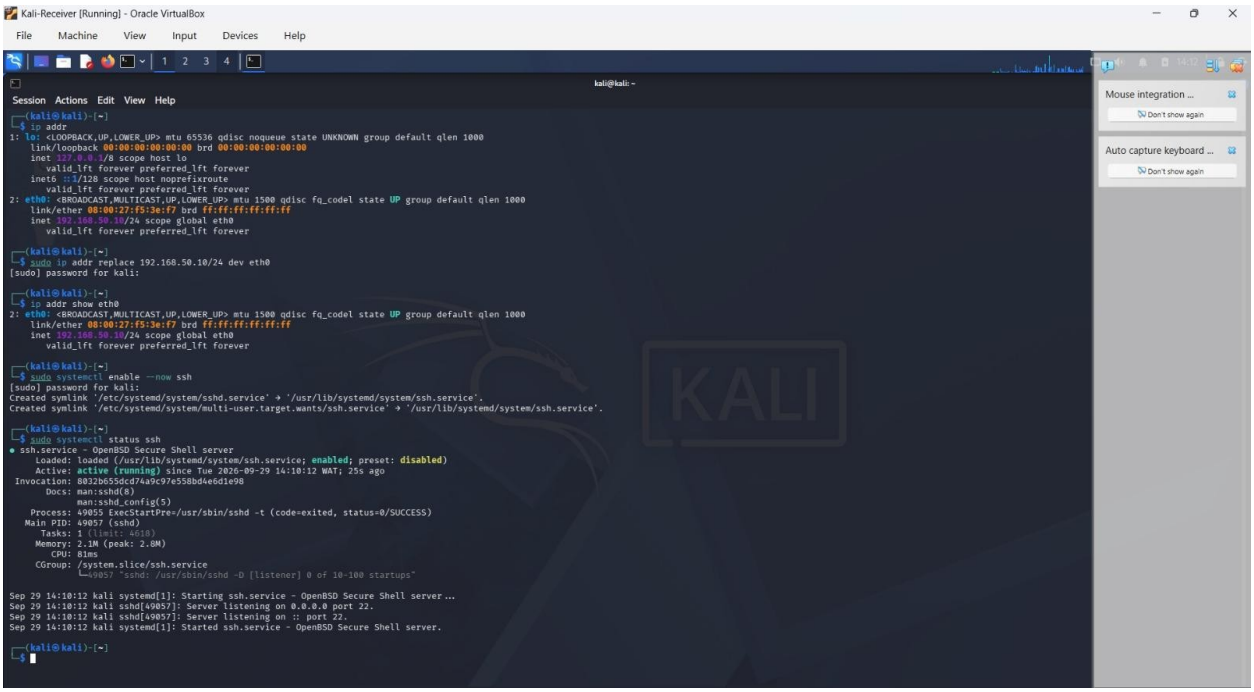
The plaintext file was encrypted using OpenSSL with the AES-256-CBC encryption algorithm. The -salt option adds a random salt to the encryption process, while -pbkdf2 specifies a password-based key derivation method for deriving the encryption key from the supplied password.

The original plaintext file was retained as sensitive_project_data.txt, while the encrypted output was saved as sensitive_project_data.txt.enc. The two files therefore represent the readable original data and its encrypted counterpart.

The purpose of this stage is to demonstrate confidentiality. Unlike the plaintext file, the encrypted file should not reveal the original message when viewed without the appropriate decryption password.

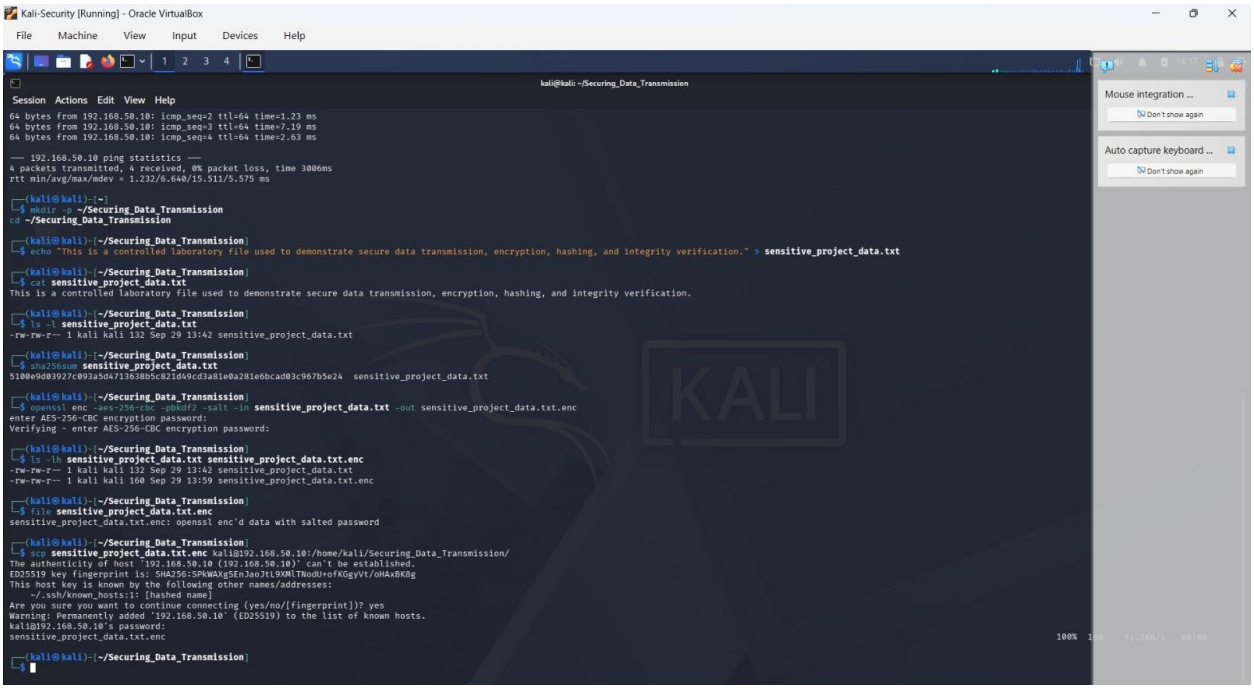
9. Secure File Transfer Using SCP

Figure 8: Verification of the SSH Service on Kali-Receiver



The SSH service was enabled and started on Kali-Receiver to provide a secure communication channel for the file transfer. The service status was checked using systemctl status ssh, confirming that the SSH server was running and ready to accept connections from the Kali-Security system.

Figure 9: Secure Transfer of the Encrypted File from Kali-Security to Kali-Receiver Using SCP



The encrypted file sensitive_project_data.txt.enc was transferred from Kali-Security (192.168.50.20) to Kali-Receiver (192.168.50.10) using the Secure Copy Protocol (SCP). The

command specifies the encrypted file as the source and the /home/kali/Securing_Data_Transmission/ directory on the receiving system as the destination.

During the first connection, SSH requested confirmation of the receiver's host key. After the host was accepted and authentication was completed, the encrypted file was successfully transferred. The SCP operation provides an authenticated and encrypted channel for transferring the protected file between the two virtual machines.

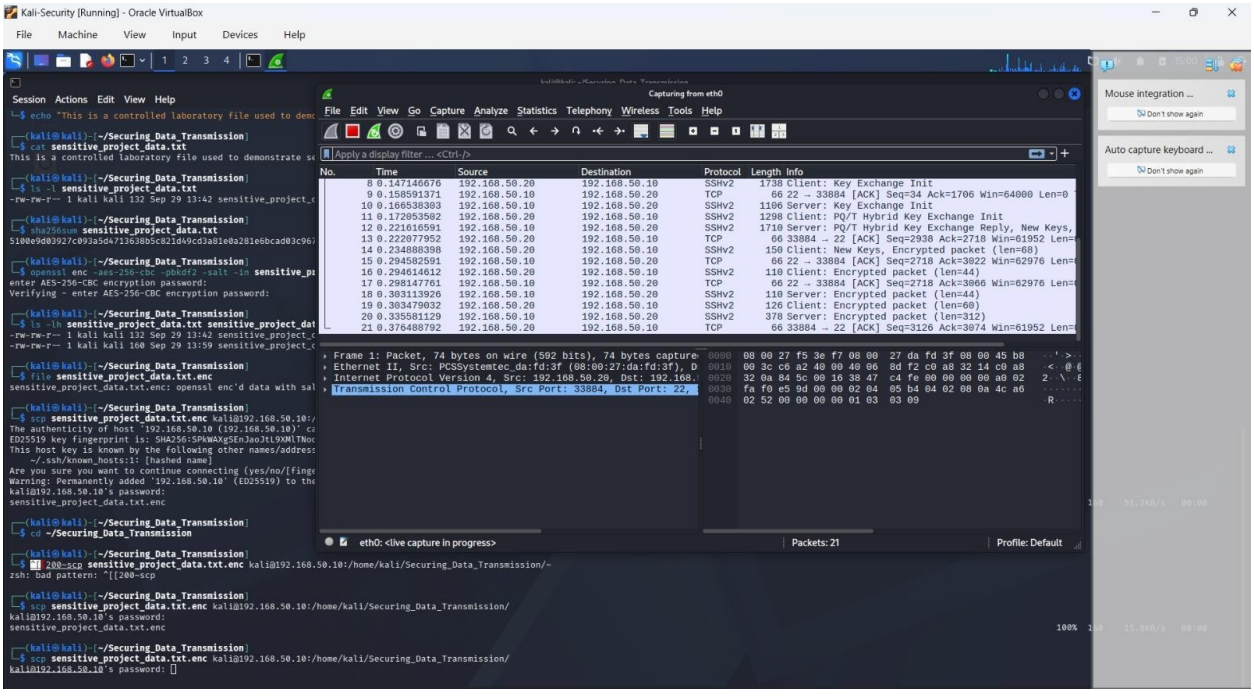
10. Wireshark Capture and Network Traffic Analysis

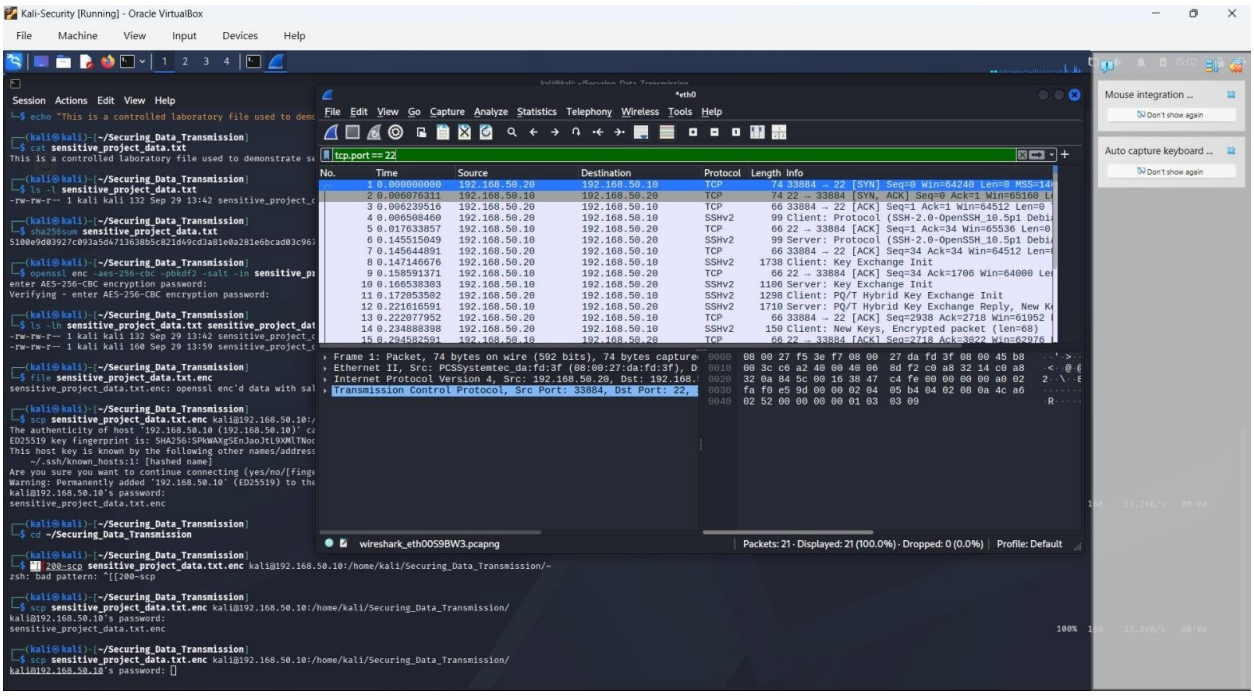
Following the successful transfer of the encrypted file from Kali-Security to Kali-Receiver using the Secure Copy Protocol (SCP), Wireshark was used to capture and analyse the network traffic generated during the transfer. The purpose of this stage was to observe the communication between the two virtual machines and determine how the encrypted file transfer appeared at the network level.

The capture was performed on the eth0 interface of Kali-Security. This interface was connected to the isolated SecureLab Internal Network and had the IPv4 address 192.168.50.20/24. Kali-Receiver was configured with the IPv4 address 192.168.50.10/24.
The Wireshark capture was started before initiating the SCP transfer so that the complete connection establishment, SSH negotiation, encrypted communication, and connection termination could be observed.

10.1 Capture of SSH Traffic

Figure 10: Wireshark Capture of SSH Traffic During SCP Transfer





Wireshark captured the network traffic generated when the encrypted file was transferred from Kali-Security to Kali-Receiver. The captured traffic shows communication between 192.168.50.20, representing Kali-Security, and 192.168.50.10, representing Kali-Receiver.

The traffic was associated with TCP port 22, which is the port used by the SSH service. Because SCP operates through SSH, the file transfer generated SSH-related traffic rather than an independent unencrypted file-transfer protocol.

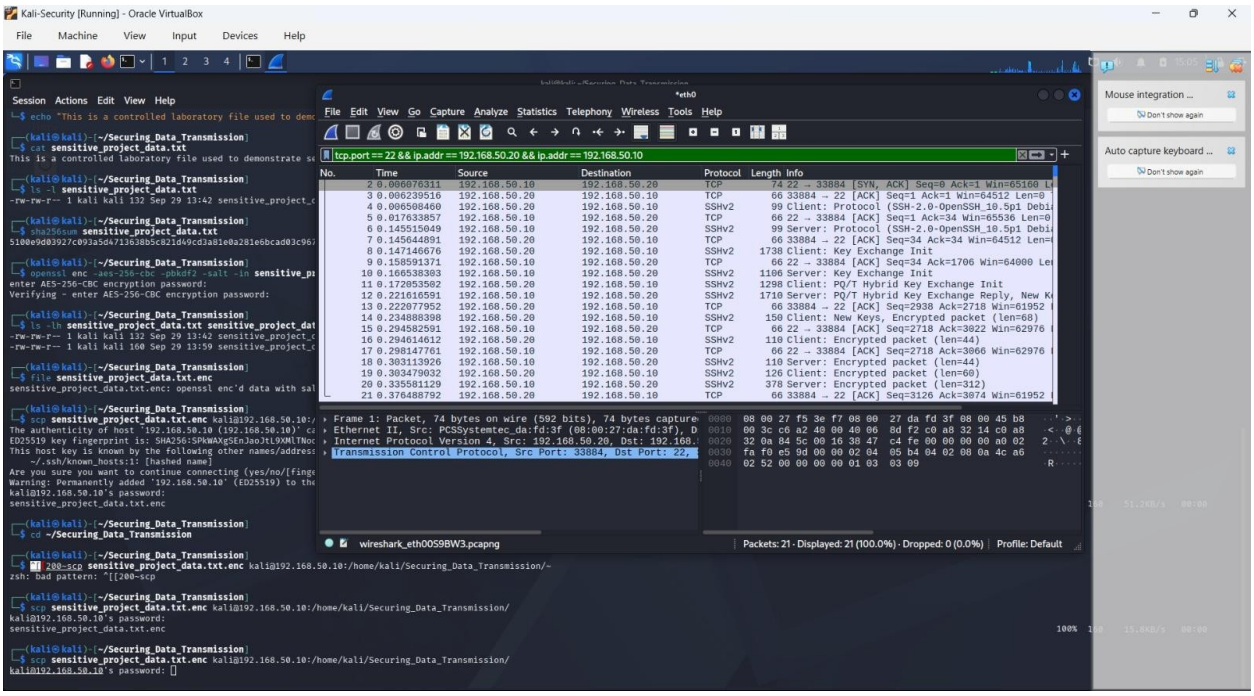
The capture contains several stages of the TCP and SSH communication. The initial packets include TCP connection-establishment packets such as SYN, SYN/ACK, and ACK. These packets establish the TCP connection between the sender and receiver before application-level communication takes place.

The capture then shows SSHv2 traffic associated with protocol negotiation and key exchange. Later packets are identified by Wireshark as encrypted SSH packets.

This demonstrates that the file transfer was not performed through an openly readable network connection. Instead, the SCP operation was carried through an SSH session.

10.2 Filtering the Communication Between the Two Virtual Machines

Figure 11: Filtered SSH Traffic Between Kali-Security and Kali-Receiver



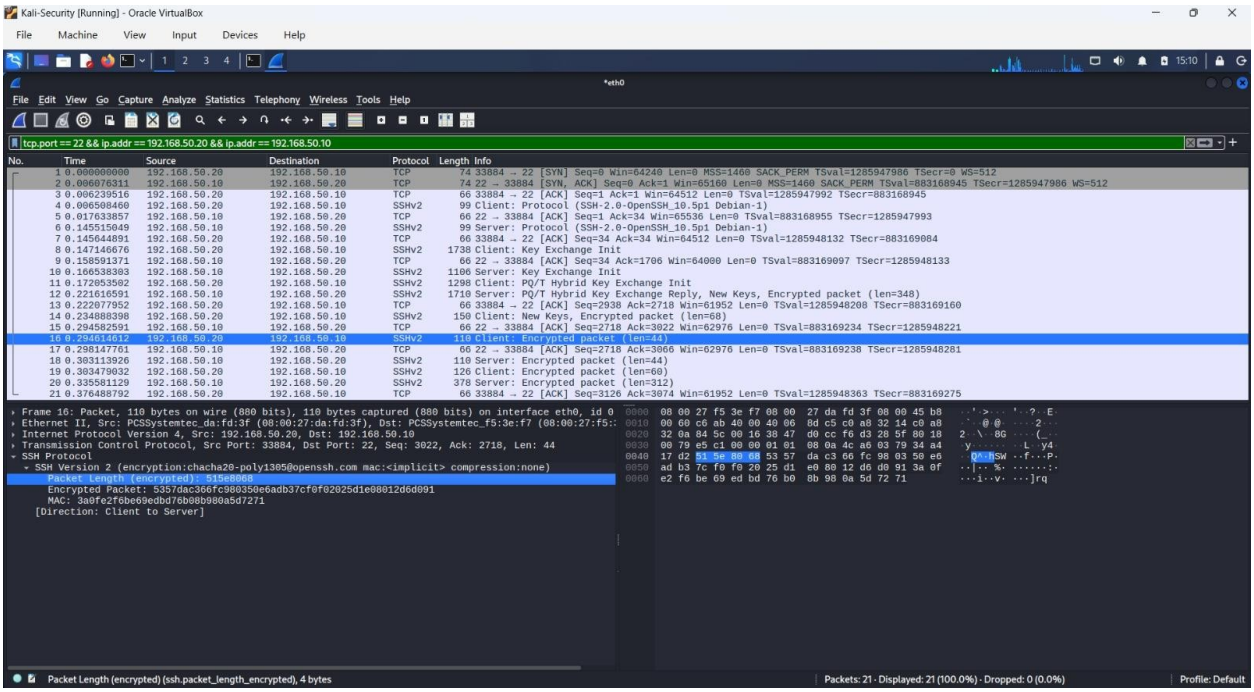
The capture contains traffic in both directions because an SSH connection requires communication between the client and server. The packets include TCP acknowledgement traffic as well as SSHv2 protocol messages.

The Wireshark capture recorded 21 displayed packets in the filtered view. The packet list includes the TCP connection establishment, SSH protocol negotiation, key-exchange activity, encrypted packets, and connection termination.

The presence of SSHv2 packets confirms that the SCP transfer was conducted through an SSH session. This is significant because SSH provides an encrypted communication channel between the sender and receiver.

10.3 Analysis of an Encrypted SSH Packet

Figure 12: Packet-Level Analysis of an Encrypted SSHv2 Packet



A representative encrypted packet was selected from the captured SSH session for more detailed analysis.

The selected packet was transmitted from Kali-Security (192.168.50.20) to Kali-Receiver (192.168.50.10) and was identified by Wireshark as an SSHv2 Client:
Encrypted packet.
The packet details show:

- Internet Protocol Version 4
- Source: 192.168.50.20
- Destination: 192.168.50.10
- TCP source port: an ephemeral client port
- TCP destination port: 22
- SSH Version 2
- Encrypted packet information
- Encryption identified by Wireshark as chacha20-poly1305@openssh.com

The selected packet is particularly important because Wireshark does not display the transferred file as readable plaintext. Instead, the SSH payload is identified as encrypted.

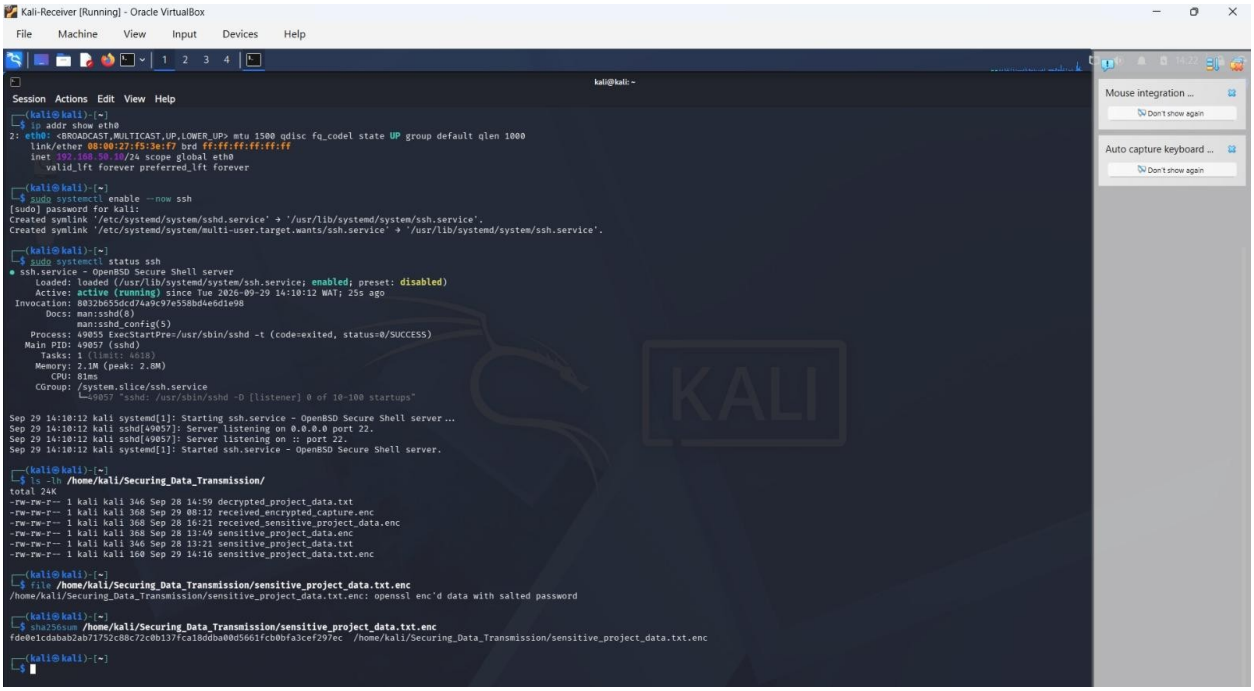
The packet therefore provides practical evidence that the communication channel used for the SCP transfer was protected by SSH encryption.

It is important to distinguish this protection from the file-level encryption performed earlier in the experiment. The original file was encrypted using AES-256-CBC with OpenSSL, while the

SSH session shown in Wireshark used the SSH transport encryption identified as ChaCha20-Poly1305. These are two separate security mechanisms operating at different stages.
The first mechanism protects the file itself, while the second protects the communication channel used to transport the file.

11. Verification of the Received Encrypted File

Figure 13: Verification of the Encrypted File Received on Kali-Receiver



The successful appearance of the file on the receiving system confirmed that the SCP transfer had completed successfully.

The file command was subsequently used to examine the received file. The output identified the file as OpenSSL encrypted data with a salted password. This was significant because it demonstrated that the transferred object remained in encrypted form after transmission.

The receiving system therefore did not simply receive a readable plaintext copy of the original file. Instead, it received the encrypted ciphertext file produced during the earlier OpenSSL encryption stage.

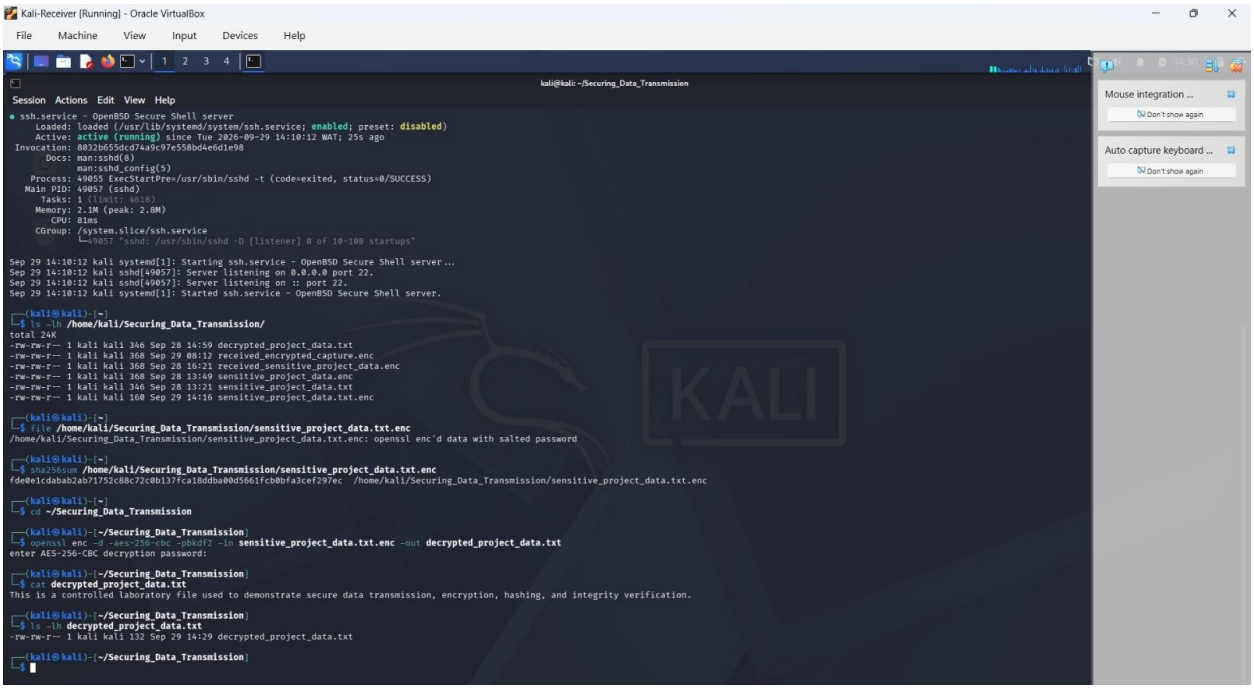
A SHA-256 checksum was also generated for the received encrypted file. This provided a cryptographic reference that could be used to compare the ciphertext before and after transmission.

The verification stage therefore established two important points:

The encrypted file successfully reached Kali-Receiver.
The received file remained encrypted after transmission.

12. Decryption of the Received File

Figure 14: Decryption of the Received AES-256-CBC Encrypted File

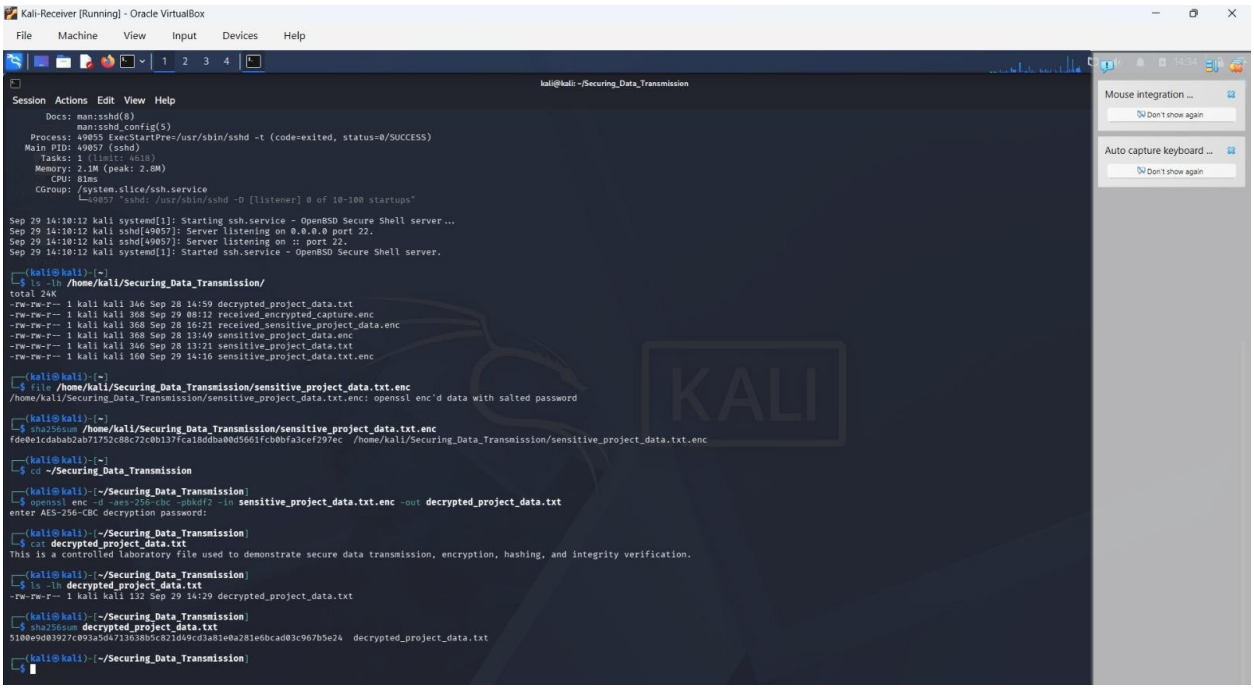


The encrypted file received by Kali-Receiver was decrypted using OpenSSL with the AES-256-CBC algorithm and the same password-based key derivation parameters used during encryption. Successful decryption demonstrated that the transmitted ciphertext could be restored to its original readable form when the correct authentication secret was supplied.

The resulting decrypted_project_data.txt file was then displayed to verify that the original laboratory message had been recovered.

13. Final Integrity Verification

Figure 12: SHA-256 Verification of the Decrypted File



The final stage of the experiment was the verification of data integrity.

A SHA-256 hash was calculated for the decrypted file using:
sha256sum decrypted_project_data.txt

The resulting SHA-256 value was compared with the SHA-256 value originally generated for the plaintext file before encryption and transmission.

The original plaintext hash had been generated using:
sha256sum sensitive_project_data.txt

The purpose of performing the comparison was to determine whether the contents of the file had remained unchanged throughout the complete process.

A cryptographic hash does not encrypt the file and does not provide confidentiality. Instead, it produces a fixed-length value that acts as an integrity reference for the contents of the file.

If the original and final SHA-256 values are identical, this indicates that the contents represented by the two files are identical for the purpose of this experiment.
The matching hash values therefore provide evidence that the original laboratory data survived the following sequence without alteration:
Plaintext creation

↓

SHA-256 reference generation
↓
AES-256-CBC encryption
↓
SCP/SSH transmission
↓
File reception
↓
AES-256-CBC decryption
↓
SHA-256 verification

The integrity check complements the confidentiality mechanisms used earlier in the experiment. Encryption protects the information from being exposed, while hashing provides a means of detecting whether the contents have changed.

14. Integrated Security Analysis

The experiment demonstrated several complementary security mechanisms rather than relying on a single security control.

14.1 Confidentiality Through File Encryption

The plaintext laboratory file was encrypted using OpenSSL and the AES-256-CBC algorithm. The encrypted output was stored as:
sensitive_project_data.txt.enc

The purpose of this stage was to ensure that the file itself was not stored or transmitted in readable plaintext form.

Without the appropriate decryption secret, the encrypted file does not directly reveal the original laboratory message.

14.2 Confidentiality During Network Transmission

The encrypted file was transferred using SCP.
SCP operates through SSH, and the Wireshark capture demonstrated that the transfer occurred through an SSHv2 session over TCP port 22.

The packet-level analysis showed encrypted SSH traffic and identified the SSH encryption mechanism used by the captured session as:
chacha20-poly1305@openssh.com

This means that the file had protection at two different levels:

File level:
AES-256-CBC
Transport level:
SSH encryption

These mechanisms serve different purposes. AES-256-CBC protects the stored/transferred file object, while SSH protects the communication channel used to transmit it.

14.3 Integrity Through SHA-256

SHA-256 was used before and after the transmission process.

The initial hash establishes a reference value for the original plaintext.

After transmission and decryption, another SHA-256 hash was calculated.

The comparison provided a method for determining whether the recovered file contents remained consistent with the original.

This demonstrates the role of cryptographic hashing in detecting unintended or unauthorized modification of data.

14.4 Role of Wireshark

Wireshark provided visibility into the network-level behaviour of the experiment.

The capture allowed the following characteristics to be observed:

- TCP connection establishment
- Communication between the two virtual machines
- TCP port 22
- SSHv2 protocol traffic
- SSH key-exchange activity
- Encrypted SSH packets
- TCP acknowledgements
- Connection termination

The capture therefore provided independent network-level evidence that the SCP operation generated SSH-protected communication between the two systems.

An important observation from the capture is that the actual file contents were not displayed as readable plaintext within the SSH packets.

15. Discussion of Results

The practical experiment successfully demonstrated the relationship between confidentiality, secure transmission, and integrity verification.

The two virtual machines were first placed on an isolated internal network, allowing communication to take place within a controlled laboratory environment.
Connectivity between the systems was verified before the file-transfer experiment began. This ensured that subsequent communication problems could be distinguished from network-configuration problems.

A controlled plaintext file was then created on Kali-Security. SHA-256 was used to generate an initial integrity reference before the file was encrypted.

The plaintext file was subsequently encrypted using OpenSSL AES-256-CBC. This created an encrypted ciphertext file that could be transferred without directly exposing the original readable content.

The encrypted file was transferred from Kali-Security to Kali-Receiver using SCP. Because SCP operates through SSH, the transfer was carried through an SSH-protected communication channel.

Wireshark was used to capture the transfer. The analysis identified SSHv2 communication between 192.168.50.20 and 192.168.50.10 over TCP port 22. The packet-level inspection identified encrypted SSH packets and showed the SSH encryption mechanism used by the captured session.

After transmission, the received encrypted file was verified on Kali-Receiver. The file command confirmed that the object remained OpenSSL-encrypted data rather than becoming plaintext during transfer.

The file was then decrypted using the appropriate AES-256-CBC parameters and password. The recovered plaintext was displayed and compared against the expected laboratory message.

Finally, SHA-256 was calculated for the recovered file and compared against the original reference. The integrity verification demonstrated whether the contents had remained unchanged throughout the experiment.
Overall, the practical exercise demonstrated how multiple security mechanisms can be combined to protect information during a data-transfer workflow.

16. Security Controls Demonstrated

Security Requirement	Technique Used	Evidence
Network isolation	Virtualbox internal network	SecureLab Configuration
Connectivity	ICMP/Ping	Successful replies between VMs
Confidentiality of file	AES-256-CBC	OpenSSL encryption
Secure transmission	SCP/SSH	TCP port 22 and SSHv2 capture
Transport encryption	SSH encryption	Wireshark encrypted SSH packets
Integrity reference	SHA-256	Original file hash
File reception	SCP	File present on Kali-Receiver
Encrypted state verification	file command	OpenSSL encrypted data identified
Data recovery	OpenSSL decryption	Original plaintext recovered
Final integrity	SHA-256 comparison	Final harsh verification

17. Limitations of the Experiment

Although the experiment successfully demonstrated the intended security mechanisms, several limitations should be recognised.

First, the experiment was conducted within a controlled virtual laboratory environment using two Kali Linux virtual machines. The environment therefore does not represent all of the complexity of a production network.

Second, the transferred file was a controlled laboratory file rather than real sensitive organisational data. This was appropriate for the experiment because it avoided exposing genuine confidential information.

Third, the Wireshark analysis was performed on the SSH traffic generated during the SCP transfer. Because the SSH session was encrypted, Wireshark could identify protocol characteristics and encrypted packets but could not display the transferred file as readable plaintext.

Finally, the experiment demonstrates the technical operation of encryption, secure transfer, and hashing, but security in a real environment would also depend on factors such as password management, host authentication, access controls, software updates, key management, and system configuration.

18. Conclusion

This project demonstrated the implementation and analysis of secure data transmission using two Kali Linux virtual machines connected through an isolated VirtualBox Internal Network.

The experiment began by establishing communication between Kali-Security and Kali-Receiver. The sender was configured with 192.168.50.20/24, while the receiver used 192.168.50.10/24.

A controlled plaintext file was created on the sender and its SHA-256 hash was generated as an integrity reference. The file was then encrypted using OpenSSL with AES-256-CBC before being transferred.

The encrypted file was transmitted using SCP over an SSH connection. Wireshark was used to capture and analyse the resulting traffic. The capture demonstrated communication over TCP port 22, SSHv2 negotiation, and encrypted SSH packet exchanges between the two virtual machines. The packet-level analysis provided evidence that the communication channel protected the transferred information rather than exposing the file contents as readable plaintext.

After the transfer, the encrypted file was verified on Kali-Receiver and subsequently decrypted using the appropriate OpenSSL parameters. The original laboratory message was successfully recovered.

A final SHA-256 calculation was then performed on the decrypted file and compared with the original integrity reference. This provided evidence for determining whether the data had remained unchanged throughout the encryption, transmission, reception, and decryption process.

The project therefore demonstrated the complementary roles of encryption, secure transport, network monitoring, and cryptographic hashing in a secure data-transmission workflow.